BAIL security

Nest
VE Core

FINAL REPORT

March '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Nest - VE Core - Audit Report
Website	usenest.xyz
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/FenixProtocol/nest-core/tree/886d90b066c06aad9f44857e5dcd91bd4a84e47b FORK: https://github.com/bailsec/nest-core/tree/886d90b066c06aad9f44857e5dcd91bd4a84e47b
Resolution 1	https://github.com/FenixProtocol/nest-core/tree/41182837fb70daac6a208e839c7dfa73c73fd9d7 FORK: https://github.com/bailsec/nest-core/tree/41182837fb70daac6a208e839c7dfa73c73fd9d7

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	1	1				
Medium	18	9	2	7		
Low	31	1		30		
Informational	68	1		67		
Governance	10			10		
Total	128	12	2	114		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Recommendation: Critical Fixes

Since the protocol is already live, making large changes to the codebase is discouraged since it would involve extensive contract upgrades. A few critical issues have been identified as must-fixes, which must be fixed with upgrades or re-deployments, and as soon as possible, since they can actively endanger funds or lead to broken operations. The procedure to deal with the rest of the issues is up to the developers, but it should be kept in mind that each fix will result in a code change which introduces risk for new issues.

The following issues should be fixed immediately:

- Issue #52 - Merge reads zeroed lock end time leading to incorrect voting weight
- Issue #53 - Checkpoint function fails to reset negative bias and slope values
- Issue #54 - compoundEmissionClaim can be temporarily DoSed
- Issue #60 - _get_y uses wrong function causing incorrect output estimates in getAmountOut
- Issue #61 - Stable pairs can be permanently broken
- Issue #68 - Stable pool mints via router can be blocked by donating tokens and syncing
- Issue #69 - Stable pool mints via router can be blocked due to equal amount check

It has to be noted that fixes must be very carefully crafted and potential upgradeability concerns should be considered, especially if the accounting mechanism or other mechanisms receive new logic.

Bribes

The Bribes system is used to incentivize voters to vote for certain gauges/pools. veNEST voters allocate vote weight to pools, and bribe contracts pay those voters rewards proportional to how they voted during an epoch (weekly accounting). There are 2 types of applicable bribes:

1. Internal bribe ("Nest LP Fees: ..."): fed by that pool's trading fees; rewards veNEST voters who directed votes to the pool.
2. External bribe ("Nest Bribes: ..."): a public incentive pot; anyone can add approved reward tokens to attract votes to that pool.

The system supports bribes in the form of ERC20 tokens as well as veNEST tokens. For veNEST tokens, a special wrapper is used to hold NEST tokens, which are then converted into locked veNEST tokens on bribe dispersal.

BribeFactoryUpgradeable

The BribeFactoryUpgradeable contract is the factory contract that deploys the Bribe implementations. On creation, it configures each bribe's allowed reward tokens: the two pool tokens (_token0, _token1 if non-zero) plus a global allowlist of defaultRewardTokens that the owner maintains. It also supports admin operations to modify the list of reward tokens allowed in the bribe contracts.

Privileged Functions

- createBribe
- setRewardClaimPause
- changeImplementation
- setVoter
- addRewards
- pushDefaultRewardToken
- removeDefaultRewardToken

Core Invariants:

INV 1: Bribe contracts can only be deployed by the voter contract or the admin

Issue_01	Global pause on bribe reward claims affects all users
Severity	**Governance**
Description	The setRewardClaimPause function in BribeFactoryUpgradeable allows the owner to pause reward claims globally across all bribe contracts. When paused, all users are blocked from claiming any bribe rewards; rewards continue to accrue but cannot be accessed, and users cannot react to market conditions. This global pause mechanism creates a severe centralization risk where a single admin action can freeze the entire bribe reward system.
Recommendations	Consider ensuring a strong governance body with a multisig.
Comments / Resolution	Acknowledged.

Issue_02	getBribeRewardTokens returns duplicate default reward tokens
Severity	**Informational**
Description	When a bribe is created via the BribeFactoryUpgradeable, the current defaultRewardToken list is immediately added to the bribe's specific rewardTokens storage: *// BribeFactoryUpgradeable.sol - createBribe()* *IBribe(newLastBribe).addRewardTokens(defaultRewardToken);* Later, when getBribeRewardTokens is called to fetch the full list of tokens for a gauge, it manually concatenates the current factory defaults with the bribe's specific list. Since the default tokens were already part of the bribeRewardsTokens array during initialization, they appear twice in the final result.
Recommendations	Consider implementing a check to ensure defaultRewardToken entries are not duplicated if they already exist in the specific bribe list.
Comments / Resolution	Acknowledged.

Issue_03	Incorrect error message in createBribe require
Severity	**Informational**
Description	The createBribe function uses a require message only voter or voter while the condition is msg.sender == voter || msg.sender == owner. This is misleading, as it suggests only the voter can call the function, but the owner is also allowed.
Recommendations	Update the require message to only voter or owner, or acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_04	Redundant zero-address check in changeImplementation
Severity	**Informational**
Description	The changeImplementation function calls _checkAddressZero(_implementation) and then immediately performs require(_implementation != address(0)). Both checks enforce the same condition (that _implementation is not the zero address), making the explicit require redundant.
Recommendations	Consider removing the redundant zero address check or acknowledge this issue.
Comments / Resolution	Acknowledged.

BribeProxy

The BribeProxy contract is a simple proxy implementation that picks up the current implementation address from the Bribe factory contract and delegates all function calls to that implementation address.

No issues were identified in this contract.

BribeUpgradeable

The BribeUpgradeable contract is the core "bribe pot" contract that tracks vote weight per epoch and streams reward tokens to the veNEST voters who supported the associated pool. This contract holds bribe reward tokens and lets voters (via the voter contract) "stake" voting power snapshots each epoch; later, veNFT owners claim a pro-rata share of each epoch's rewards based on their recorded vote weight.

Rewards are keyed by (_rewardToken, epochStart) in rewardData, and the contract uses the minter.active_period as the epoch start. New rewards are added to the current epoch bucket via notifyRewardAmount, with periodFinish = epochStart + WEEK.

Users can claim their allocated rewards by calling the getReward function.

Appendix: Deposit/Withdraw

Whenever a user votes with their ve NEST token in the Voter contract, the contract, in turn, calls the deposit function on the bribe contracts. This does not involve any actual token transfers and is instead just a function to snapshot the current voting balance of the tokenId. This value is then stored in the bribe contract and is used to dispense rewards to the owner of the veNEST token.

Appendix: Bribe calculation

The balance of each owner for each epoch is stored in the _balances[_owner][_startTimestamp] mapping. When notifyRewardAmount is called by the Gauge or users, the contract also stores the per-epoch bribes to be distributed in the rewardData[_rewardsToken][_startTimestamp] mapping. Then, the actual bribe is calculated as

> rewardData[_rewardsToken][_timestamp].rewardsPerEpoch / _totalSupply[_timestamp] x _balance

This calculation is done for each epoch to find the total amount of bribe owed to any user.

Privileged Functions

- addRewardToken
- recoverERC20AndUpdateData
- emergencyRecoverERC20
- setVoter
- setMinter

Core Invariants:

INV 1: Only allowed tokens can be used as bribes. Either allowed specifically or part of the default allowed tokens group.
INV 2: _totalSupply[t] is intended to equal the sum of all _balances[addr][t] over addresses for that epoch.
INV 3: Rewards for the same epoch can never be claimed twice.
INV 4: At most 50 epochs are processed per call, so "fully caught up" is not guaranteed if the user is >50 epochs behind.

Issue_05	Emergency recover function can break user claims in BribeUpgradeable
Severity	**Governance**
Description	The BribeUpgradeable contract has an emergencyRecoverERC20 function that allows admins to withdraw any ERC20 tokens from the contract. While intended for emergency situations, this function can be misused to drain funds that users are entitled to claim. Users who have earned bribe rewards but haven't claimed yet will be unable to claim, as the accounting won't be updated, and claims will fail.
Recommendations	Consider ensuring a strong governance body with a multisig.
Comments / Resolution	Acknowledged.

Issue_06	Unused storage mapping userRewardPerTokenPaid
Severity	**Informational**
Description	The BribeUpgradeable contract defines the mapping userRewardPerTokenPaid, but never reads from or writes to it anywhere in the contract. This unused storage increases the contract's storage footprint and can confuse readers, as it suggests a reward-per-token accounting mechanism that is no longer in use. Since the contract has already been deployed, the mapping cannot be removed without an upgrade, but it can be treated as deprecated.
Recommendations	Remove the unused code or acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_07	Unused whenRewardClaimPaused modifier
Severity	**Informational**
Description	The BribeUpgradeable contract defines the whenRewardClaimPaused modifier, but this modifier is never applied to any function. Its logic duplicates the inverse of whenNotRewardClaimPaused and checks the same isRewardClaimPause flag from the factory, yet it is unused in the current implementation. Keeping an unused modifier adds noise and can confuse readers, suggesting behavior (functions gated on the paused state) that does not actually exist.
Recommendations	Remove the unused code or acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_08	balanceOf / balanceOfOwner always returns 0 for the next epoch and is effectively dead code
Severity	**Informational**
Description	In BribeUpgradeable, the functions: - balanceOf(uint256 tokenId) - balanceOfOwner(address _owner) compute balances for the **next** epoch by using getNextEpochStart as the timestamp key: - uint256 _timestamp = getNextEpochStart(); - _balances[_owner][_timestamp] However, in the current implementation, deposits are only ever recorded for the **current** epoch timestamp, never for a future epoch. There is no code path that adds balances at getNextEpochStart, so these functions will always return 0.
Recommendations	Remove the dead code or acknowledge this issue.
Comments / Resolution	Acknowledged.

BribeVeNESTRewardToken

The BribeVeNESTRewardToken contract implements an intermediary "bribe token" called brVeNEST that is backed 1:1 by the protocol's underlying token (underlyingToken, i.e., NEST) and is designed to turn bribe payments into veNEST locks automatically.

A minter (typically a router like CustomBribeRewardRouter) can mint brVeNEST by depositing the underlying NEST into this contract (mint(to, amount) transfers NEST in, then mints brVeNEST out). When brVeNEST is later transferred to a normal user address, the token's transfer hook _afterTokenTransfer burns the received brVeNEST and uses the underlying NEST held by this contract to call VotingEscrow.createLockFor(...), minting a veNEST position for the recipient.

Privileged Functions

- updateCreateLockParams
- mint

Core Invariants:

INV 1: Total brVeNEST supply corresponds to the underlying NEST held by this contract minus any underlying already used to create locks.
INV 2: Only accounts with MINTER_ROLE can mint brVeNEST.
INV 3: No conversion when the recipient is whitelisted. Otherwise, on any transfer to a non-whitelisted address from a non-minter, the contract must burn the amount from the recipient and create a new lock of the same amount for the receiver.

Issue_09	Use of deprecated safeApprove
Severity	**Informational**
Description	The BribeVeNESTRewardToken contract uses safeApprove when approving underlyingToken for votingEscrow: - IERC20Upgradeable(underlyingToken).safeApprove(address(votingEscrowCache), amount_); In the OpenZeppelin version used by this project, safeApprove is deprecated because it can revert when changing a non-zero allowance to another non-zero value. This pattern is fragile and can break if the allowance is not first reset to zero. The same issue is also present in other contracts throughout the project, and all such instances should be fixed.
Recommendations	Consider replacing safeApprove with forceApprove (or an equivalent pattern that first sets the allowance to zero before setting the new value), or acknowledge this issue.
Comments / Resolution	Fixed by following recommendation.

CustomBribeRewardRouter

The CustomBribeRewardRouter contract is a routing contract for funding external bribes using NEST, by converting NEST (or NEST reclaimed from a veNEST NFT) into the intermediary token brVeNEST, then notifying the correct external bribe contract for a given pool. The contract has 2 supported funding paths:

1. notifyRewardNESTInVeNEST: This pulls underlying NEST from caller -> approves/mints brVeNEST -> notifyRewardAmount to the bribe contract.
2. notifyRewardVeNESTInVeNest: This takes custody of a veNEST NFT (ERC721) from caller -> burns it to unlock the underlying tokens -> mints brVeNEST for that amount -> notifies the external bribe.

In both cases, the Bribe contract is funded with brVeNEST tokens. When these tokens are dispensed to the users, they are automatically burnt, and the underlying NEST tokens are instead locked, and the resulting veNEST tokens are sent to the receiver address.

Privileged Functions

- setupFuncEnable

Core Invariants:

INV 1: Calls to notifyRewardNESTInVeNEST and notifyRewardVeNESTInVeNest must revert unless their respective selector is enabled in funcEnabled.
INV 2: No rewards are “created from nothing”: the brVeNEST minted corresponds to the underlying NEST acquired either from the caller (direct path) or from burnToBribes (NFT path).

Issue_10	Admin can enable/disable any function in CustomBribeRewardRouter
Severity	**Governance**
Description	The setupFuncEnable function in CustomBribeRewardRouter allows the admin to enable or disable any function selector at will. This gives the admin complete control over the contract's functionality, including the ability to disable legitimate functions, break the reward distribution flow, or block user access to claiming rewards. This level of control over function execution represents a significant centralization risk.
Recommendations	Consider ensuring a strong governance body with a multisig.
Comments / Resolution	Acknowledged.

Issue_11	Redundant permanent-lock check and unlock before calling burnToBribes
Severity	**Informational**
Description	In CustomBribeRewardRouter.notifyRewardVeNESTInVeNest, the contract performs an explicit check and unlock of a permanently locked veNFT: *if (votingEscrow.getNftState(tokenId_).locked.isPermanentLocked) {* *votingEscrow.unlockPermanent(tokenId_);* *}* *votingEscrow.burnToBribes(tokenId_);* However, the VotingEscrowUpgradeableV2.burnToBribes implementation already calls _unlockPermanentIfLocked(tokenId_) internally, handling the permanent-lock removal itself. As a result, the explicit unlockPermanent call in CustomBribeRewardRouter is redundant and adds an extra external call and storage read without changing behavior.
Recommendations	Remove the redundant checks or acknowledge this issue.
Comments / Resolution	Acknowledged.

Core

Libraries/DateTime

The DateTime library is a gas-efficient utility library for date and time operations in Solidity. It is a copy of BokkyPooBah's DateTime Library and provides functions for converting between Unix timestamps and human-readable date components, as well as arithmetic operations on dates.

Libraries/LibVoterErrors

The LibVoterErrors library defines custom error types used by the VoterUpgradeableV2 contract and related voting infrastructure. Custom errors provide gas-efficient revert reasons compared to string-based require statements and enable more precise error handling by external callers.

Libraries/LibVotingEscrowConstants

The LibVotingEscrowConstants library defines time-related constants used throughout the VotingEscrow system for lock duration calculations and epoch management.

Libraries/LibVotingEscrowErrors

The LibVotingEscrowErrors library defines custom error types used by the VotingEscrowUpgradeableV2 contract and related veNFT infrastructure. These errors provide gas-efficient reverts with descriptive failure conditions for the vote-escrow system.

Libraries/LibVotingEscrowUtils

The LibVotingEscrowUtils library provides utility functions used throughout the VotingEscrow system for time calculations and type conversions. It imports the constants from LibVotingEscrowConstants to access WEEK and MAX_LOCK_TIME values.

Libraries/NumberFormatter

The NumberFormatter library provides utility functions for converting numeric values into human-readable string representations with formatting. It is primarily used for generating display strings in NFT metadata, such as the veNFT art proxy that shows locked token amounts.

Issue_12	Malformed string output when limitFractionNumbers is 0
Severity	**Informational**
Description	In NumberFormatter.formatNumber, the decimal point . is hardcoded into the abi.encodePacked operation. The function fails to account for cases where limitFractionNumbers is set to 0. In such cases, the limitFactionNumbers helper returns an empty string, but the preceding dot remains in the final output.
Recommendations	Add a conditional check to only append the fractional components if limitFractionNumbers is greater than zero or acknowledge this issue.
Comments / Resolution	Acknowledged.

CompoundEmissionExtensionUpgradeable

The CompoundEmissionExtensionUpgradeable contract is an extension built on top of the Voter contract that automates the compounding of user emissions into veNFT locks and/or external bribe pools.

Users configure what percentage of their claimed emissions should be allocated to veNFT locks versus external bribe pools via setCompoundEmissionConfig. Each allocation can be further split across multiple targets: for locks, users specify veNFT token IDs (or 0 to create new locks); for bribes, users specify pool addresses. The target percentages within each category must sum to exactly 100%. When compounding occurs, the contract claims emissions from Voter on behalf of the user, then routes the tokens according to their configuration. If a target bribe pool's gauge has been killed, the emissions that would have gone there are redirected to create a new veNFT lock instead. A keeper with COMPOUND_KEEPER_ROLE can batch compound for multiple users via compoundEmissionClaimBatch, or users can trigger their own claims directly via compoundEmisisonClaim.

Privileged Functions

- compoundEmissionClaimBatch
- setDefaultCreateLockConfig
- changeEmissionTargetLockId

Core Invariants:

INV 1: A user's total allocation cannot exceed 100% (getToLocksPercentage + getToBribePoolsPercentage <= 1e18).
INV 2: If a user has a non-zero lock percentage, they must have at least one target lock configured, and vice versa. Same rule applies for bribe pool percentage and target pools.
INV 3: Sum of all TargetLock percentages must equal exactly 1e18 when set, and sum of all TargetPool percentages must equal exactly 1e18 when set.
INV 4: Users can only configure target locks for veNFTs they own. Target pools must have an alive gauge at configuration time.
INV 5: If withPermanentLock is false in any CreateLockConfig, then lockDuration must be greater than zero.

Issue_13	safeApprove usage leads to DoS due to leftover allowance from rounding errors
Severity	**Medium**
Description	The compoundEmission function uses the deprecated safeApprove` function which reverts when trying to set a non-zero allowance while a non-zero allowance already exists. The function approves toTargetLock upfront, then distributes it across multiple target locks using amount = (targetLock.percentage * toTargetLocks) / _PRECISION. Due to rounding down in the division, the sum of all distributed amounts will be less than or equal to toTargetLocks, leaving leftover allowance that remains unused. On subsequent calls to compoundEmission, safeApprove will revert because it attempts to set a non-zero allowance when there's already leftover allowance from the previous call, causing a denial of service. Additionally, safeApprove is deprecated in OpenZeppelin 4.9.3. The contract also uses safeApprove in other locations, but those are less critical as they approve exact amounts that will be fully consumed, reducing the likelihood of leftover allowance.
Recommendations	Replace safeApprove with forceApprove.
Comments / Resolution	Fixed by following recommendation.

Issue_14	Compounding reverts on zero-amount allocations
Severity	**Low**
Description	A user's reward allocation can round down to zero if their total reward is small or if they have split their percentages across many targets. The CompoundEmissionExtensionUpgradeable contract attempts to process these allocations without checking if the amount is greater than zero. The downstream VotingEscrowUpgradeableV2 contract contains explicit reverts for zero-value deposits. As a result, any user configuration that results in a 0 amount for a single target will cause the entire compoundEmissionClaim call to fail.
Recommendations	Implement a conditional check *if (amount > 0)* before calling any external deposit or lock functions in the compounding loop or document and acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_15	Dead gauge fallback ignores user configured tokenId and creates unnecessary new locks
Severity	**Low**
Description	The _compoundEmissionClaim function handles two types of emission targets: target locks and target bribe pools. When processing target locks, the code correctly checks if the user has an existing tokenId configured and calls depositFor to add to that existing veNFT. However, when processing target bribe pools and encountering a dead gauge, the fallback logic always creates a new lock instead of checking if the user already has a configured tokenId they want to use. In the toTargetLocks path, the code properly checks the tokenId: *if (targetLock.tokenId == 0) {* *targetLock.tokenId = votingEscrowCache.createLockFor(...);* *_usersCompoundEmissionTargetLocks[claimParams_.target][i].tokenId = targetLock.tokenId;* *} else {* *votingEscrowCache.depositFor(targetLock.tokenId, amount, false, false);* *}* In the toTargetBribePools path when the gauge is not alive, the code unconditionally creates a new lock: *if (voterCache.isAlive(gauge)) {* *// Send to bribe* *} else {* *uint256 tokenId = votingEscrowCache.createLockFor(...);* *}* The user may have already configured their preferred tokenId in the _usersCompoundEmissionTargetLocks mapping. This configuration indicates where they want their compounded emissions to go. When

	a gauge dies and the fallback triggers, this preference is ignored and a new veNFT is created instead. Consider a user who has configured tokenId 123 in their target locks and also has several bribe pools configured. If one of those pools has its gauge killed, the fallback creates a brand new veNFT rather than depositing into tokenId 123. Over time this leads to the user accumulating fragmented veNFT positions they did not request.
Recommendations	Before creating a new lock in the dead gauge fallback, check if the user has any existing tokenId configured in their _usersCompoundEmissionTargetLocks. If a non-zero tokenId exists, call depositFor on that tokenId instead of creating a new lock. Or document this behaviour and acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_16	In case a gauge is killed _compoundEmissionClaim will create a locked NFT for the user instead of reverting
Severity	**Informational**
Description	Token emissions can be compounded to gauges or new locks. In case a user configures a gauge that is killed later, the next compound will create a lock instead of forcing the user to change his configurations. This might be confusing.
Recommendations	Revert in case the gauge is not alive anymore to allow the user to change his configurations.
Comments / Resolution	Acknowledged.

Issue_17	Typo in function name compoundEmisisonClaim
Severity	**Informational**
Description	The CompoundEmissionExtensionUpgradeable contract contains a typo in the function name compoundEmisisonClaim where "Emisison" should be "Emission".
Recommendations	Correct the function name from compoundEmisisonClaim to compoundEmissionClaim to use proper spelling.
Comments / Resolution	Acknowledged.

GaugeRewarder

The GaugeRewarder contract manages off-chain computed reward distributions to gauges and handles signature-based claiming for users. It serves as a flexible reward system where reward amounts can be calculated off-chain and distributed based on EIP-712 signed messages.

Gauges or accounts with REWARDER_ROLE can notify rewards for a specific gauge by calling notifyReward or notifyRewardWithTransfer. The contract tracks rewards per gauge per epoch via rewardPerGaugePerEpoch and maintains accounting of total distributed versus claimed amounts. Users claim their accumulated rewards by providing a valid EIP-712 signature from the designated signer, which includes their address, total claimable amount, and a deadline. The signature allows for incremental claiming where users can claim the difference between their new total allocation and what they have already claimed. Accounts with CLAMER_FOR_ROLE can claim on behalf of other users via claimFor.

Privileged Functions

- setSigner
- notifyReward
- notifyRewardWithTransfer
- claimFor

Core Invariants:

INV 1: totalRewardDistributed >= totalRewardClaimed. Cannot claim more than has been distributed.
INV 2: claimed[user] <= totalAmount in signature. Users can only claim up to their signed allocation.
INV 3: Claims are disabled when signer is set to address zero.
INV 4: Signature must not be expired (deadline > block.timestamp) for a claim to succeed.
INV 5: Available balance for new distributions equals token balance minus (totalRewardDistributed - totalRewardClaimed).

Issue_18	Changing signer in GaugeRewarder can allow unauthorized fund drainage
Severity	**Governance**
Description	The GaugeRewarder contract relies on an off-chain signer to authorize reward claims via EIP-712 signatures. The admin can change the signer address at any time using setSigner. If a malicious or compromised admin sets the signer to an address they control, the new signer can generate valid signatures for any claim amount to arbitrary addresses, effectively draining all reward tokens from the contract.
Recommendations	Add a timelock for signer changes to allow users to claim pending rewards before the change takes effect.
Comments / Resolution	Acknowledged.

Issue_19	Attacker can permanently DoS the GaugeRewarder
Severity	**Low**
Description	The GaugeRewarder can be permanently DOSed in the following way: Say 2 users Alice and Bob have valid merkle claims of 100 tokens each. For now there are no reward tokens notified in the contract. Alice donates 100 tokens directly to the contract and calls claim for her 100 tokens. totalRewardsClaimed is now 100 and she gets back her donated 100 tokens. But this creates an issue since when rewards would be notified to the contract i.e. say 100 tokens are being notified then the following line *IERC20Upgradeable(token).balanceOf(address(this)) - (totalRewardDistributed - totalRewardClaimed)* would keep reverting because the bracket portion underflows and it would always keep reverting since totalRewardDistribution would never be updated i.e. remains 0 and total claimed is non zero.
Recommendations	Consider not producing valid merkle claims before those rewards are notified in the contract or consider documenting and acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_20	Unused import PausableUpgradeable
Severity	Informational
Description	The GaugeRewarder contract imports PausableUpgradeable from OpenZeppelin but does not use it. The contract does not inherit from PausableUpgradeable and does not implement any pause functionality.
Recommendations	Remove the unused import to reduce contract size and improve code clarity.
Comments / Resolution	Acknowledged.

Issue_21	Typo in role constant name _CLAMER_FOR_ROLE
Severity	Informational
Description	The GaugeRewarder contract contains a typo in the role constant name _CLAMER_FOR_ROLE where "CLAMER" should be "CLAIM". This typo appears in the constant definition and is used in the onlyRole(_CLAMER_FOR_ROLE) modifier.
Recommendations	Correct the constant name from _CLAMER_FOR_ROLE to _CLAIM_FOR_ROLE and update the keccak256 hash accordingly to use proper spelling.
Comments / Resolution	Acknowledged.

Issue_22	Redundant external call to active_period in event emission
Severity	**Informational**
Description	In the notifyRewardAmount function, the epoch is already fetched and stored in a local variable epoch on line 230. However, in the event emission on line 236, IMinter(minter).active_period is called again instead of using the stored epoch variable. This results in an unnecessary external call that wastes gas.
Recommendations	Use the stored epoch variable in the event emission.
Comments / Resolution	Acknowledged.

MinterUpgradeable

The MinterUpgradeable contract is the emission controller for the NEST token following ve(3,3) tokenomics. It is responsible for minting new NEST tokens on a weekly basis and distributing them to the Voter contract for gauge rewards.

The emission schedule has three phases. First, the contract must be activated by the owner calling start. During the initial inflation period (12 weeks by default), weekly emissions increase by the inflation rate (1.5%) each week. After the inflation period ends, emissions enter the decay phase where they decrease by the decay rate (1%) weekly. However, emissions never fall below the tail emission floor, which is 0.2% of the circulating supply (total supply minus tokens locked in VotingEscrow). When update_period is called and a new week has begun, the contract calculates the weekly emission, deducts the team rate (up to 5%) which is sent to the owner, and transfers the remainder to the Voter contract via notifyRewardAmount. The contract will mint new NEST tokens if its balance is insufficient to cover the weekly distribution.

Privileged Functions

- start
- setVoter
- setVotingEscrow
- setTeamRate
- setDecayRate
- setInflationRate

Core Invariants:

INV 1: update_period can only mint new emissions once per week (epoch-based on Thursday 00:00 UTC).
INV 2: teamRate cannot exceed MAX_TEAM_RATE (500 bips / 5%).
INV 3: Weekly emissions floor is max(decayed_weekly, circulating_emission) where circulating_emission is 0.2% of circulating supply.
INV 4: decayRate and inflationRate cannot exceed PRECISION (10000 / 100%).
INV 5: Emissions only begin distributing after isStarted is true and startEmissionDistributionTimestamp has passed.

Issue_23	Uninitialized startEmissionDistributionTimestamp
Severity	**Informational**
Description	The MinterUpgradeable contract contains an uninitialized state variable startEmissionDistributionTimestamp which defaults to 0. This results in the emission logic being permanently active as soon as the minter is started, potentially bypassing a planned protocol launch delay. The update_period function uses startEmissionDistributionTimestamp as a gate for minting new weekly emissions. However, since the variable is never assigned a value in the initialize function and no setter exists, the condition block.timestamp >= startEmissionDistributionTimestamp is always true.
Recommendations	Assign a value to startEmissionDistributionTimestamp in the initialize function or provide an onlyOwner setter.
Comments / Resolution	Acknowledged.

Nest

The Nest contract is the core ERC20 protocol token with a fixed initial supply of 1 billion tokens minted to the deployer at construction. Ownership is immediately transferred to the minter address (typically the MinterUpgradeable contract), granting exclusive rights to mint additional tokens for weekly emissions.

The contract extends OpenZeppelin's ERC20Burnable, allowing any holder to burn their own tokens, and Ownable for access control on minting.

Privileged Functions

- mint

Core Invariants:

INV 1: Only the owner (Minter contract) can mint new tokens.
INV 2: Initial supply is exactly 1,000,000,000 NEST (1e27 wei) minted to deployer.

Issue_24	Incorrect dev comment references non-existent EmmisionManager contract
Severity	**Informational**
Description	The dev comment in the Nest contract incorrectly references EmmisionManager as the contract that can mint tokens. However, there is no such contract in the codebase. The only entity that can mint tokens is the MinterUpgradeable contract, which becomes the owner of the Nest contract and has access to the mint function through the onlyOwner modifier.
Recommendations	Update the dev comment to correctly reference MinterUpgradeable instead of the non-existent EmmisionManager contract.
Comments / Resolution	Acknowledged.

NestRaiseUpgradeable

The NestRaiseUpgradeable contract manages a two-phase token raise consisting of a whitelist phase followed by a public phase. Users deposit a specified token and later claim reward tokens split between liquid tokens and veNFT locks.

During the whitelist phase, only users who can provide a valid Merkle proof can deposit. Whitelisted users may have individual deposit caps encoded in the Merkle tree, or they fall back to the default whitelist phase cap. During the public phase, any user can deposit up to the public phase cap. Both phases enforce a global total deposit cap. After the raise ends and the claim phase begins, depositors call claim to receive their reward tokens. The reward amount is calculated based on the deposit amount multiplied by amountOfRewardTokenPerDepositToken. This reward is then split according to toVeNftPercentage: that percentage is locked into a veNFT for 182 days via VotingEscrow.createLockFor, and the remainder is transferred as liquid tokens. Users can only claim once. After the raise ends, the owner can withdraw collected deposits to the depositsReciever address and recover any unclaimed reward tokens.

Privileged Functions

- setDepositCaps
- setWhitelistRoot
- setTimestamps
- withdrawDeposits
- withdrawExcessiveRewardTokens

Core Invariants:

INV 1: Phase timestamps must be sequential: startWhitelistPhaseTimestamp < startPublicPhaseTimestamp < endPublicPhaseTimestamp < startClaimPhaseTimestamp.
INV 2: User deposits cannot exceed the phase-specific cap (whitelist cap during whitelist phase, public cap during public phase).
INV 3: Total deposits across all users cannot exceed totalDepositCap.
INV 4: toVeNftPercentage cannot exceed 1e18 (100%).
INV 5: Each user can only claim once (isUserClaimed mapping enforced).
INV 6: Owner can only withdraw deposits and excess reward tokens after endPublicPhaseTimestamp has passed.

Issue_25	Donation attack prevents owner from withdrawing excess reward tokens
Severity	**Medium**
Description	The withdrawExcessiveRewardTokens function calculates excess rewards based on the current balance of deposit tokens in the contract rather than using the tracked totalDeposited variable. This allows anyone to donate deposit tokens to the contract and artificially inflate the calculated reward obligations, preventing the owner from recovering unused reward tokens. The contract properly tracks actual deposits in a state variable: *uint256 public override totalDeposited;* *// In deposit function:* *totalDeposited += amount_;* However withdrawExcessiveRewardTokens uses balanceOf instead: *(uint256 toRewardTokenAmount, uint256 toVeNftAmount) = getRewardsAmountOut(* *IERC20Upgradeable(token).balanceOf(address(this))* *);* *uint256 unclaimedRewards = balance - (toRewardTokenAmount + toVeNftAmount - totalClaimed);* The getRewardsAmountOut function calculates the reward token amounts that should be distributed based on the deposit amount passed in. When the deposit token balance is artificially increased through donations, the calculated reward obligations increase proportionally. Consider a scenario where a raise concludes with 1000 deposit tokens collected and the contract holds 500 reward tokens with 300 already claimed. The owner expects to withdraw roughly 100 excess tokens. An attacker donates 1000 deposit tokens directly to the contract. Now balanceOf returns 2000 while totalDeposited

	remains 1000. The getRewardsAmountOut calculation doubles the expected reward distribution. When computing unclaimedRewards, the inflated obligation causes the subtraction to underflow and revert, or results in zero which also reverts due to the _checkAmountZero validation. The attacker loses their donated tokens but successfully prevents the owner from ever recovering excess reward tokens from the contract. Furthermore, if withdrawExcessiveRewardTokens is called after the withdrawDeposits function, this actually removes all pending rewards from the contract as well, leaving the balance as 0.
Recommendations	Replace the balanceOf call with the totalDeposited state variable which accurately reflects actual user deposits and cannot be manipulated through direct token transfers. This doesnt pose an immediate threat since the contract is not being used right now.
Comments / Resolution	Acknowledged.

Issue_26	Deposit caps in NestRaise can be bypassed
Severity	**Low**
Description	In NestRaise if the token being raised is a callback token then the totalDepositCap and user deposit cap can be bypassed, since the transfer of token is performed before updating userDeposited and totalDeposited an attacker could reenter the deposit function and the deposit cap checks would pass every time since the userDeposited and totalDeposited still store the old deposited amount and then at the end of execution they both will be updated to the inflated values.
Recommendations	Perform the transfer of token after updating userDeposited and totalDeposited.
Comments / Resolution	Acknowledged.

Issue_27	Inconsistent inclusion check in isClaimPhase
Severity	**Informational**
Description	The NestRaiseUpgradeable contract uses inclusive checks for isWhitelistPhase and isPublicPhase. However, the isClaimPhase function deviates from this pattern and uses an exclusive check. As a result, isClaimPhase returns false when the block.timestamp equals the startClaimPhaseTimestamp.
Recommendations	Change the comparison operator to >= to align with the logic used in isWhitelistPhase and isPublicPhase.
Comments / Resolution	Acknowledged.

Issue_28	Multiple typos in naming conventions throughout the NestRaise contract
Severity	**Informational**
Description	The NestRaiseUpgradeable contract contains several typos in naming conventions that should be corrected for consistency and clarity: 1. "Reciever" instead of "Receiver" 2. "whithdraw" instead of "withdraw" 3. "balanace" instead of "balance"
Recommendations	Correct all typos to use proper spelling.
Comments / Resolution	Acknowledged.

Issue_29	Redundant timestamp validation check
Severity	**Informational**
Description	In the setTimestamps function, the check startWhitelistPhaseTimestamp_ >= endPublicPhaseTimestamp_ is redundant. The function already validates that startWhitelistPhaseTimestamp_ < startPublicPhaseTimestamp_ and startPublicPhaseTimestamp_ < endPublicPhaseTimestamp_. By transitivity, if both of these conditions are satisfied, then startWhitelistPhaseTimestamp_ < endPublicPhaseTimestamp_ must also be true, making the direct check unnecessary.
Recommendations	Remove the redundant check startWhitelistPhaseTimestamp_ >= endPublicPhaseTimestamp_ from the validation logic to reduce gas costs while maintaining the same validation guarantees.
Comments / Resolution	Acknowledged.

Rnest

The RNest contract implements a redeemable token (rNEST) that is backed 1:1 by the protocol's underlying NEST token and designed to convert into a fixed split of liquid NEST and locked veNEST.

The owner (typically a protocol admin or distributor) can mint rNEST by calling mint, which creates rNEST tokens without requiring NEST deposit at mint time. The contract must be pre-funded with NEST to cover conversions. When a user calls convert or convertAll, their rNEST is burned and the contract distributes the underlying NEST as follows: 40% is sent directly to the user as liquid NEST, and the remaining 60% is used to call VotingEscrow.createLockFor, creating a veNEST position locked for 182 days for the caller.

Privileged Functions

- mint
- recoverToken

Core Invariants:

INV 1: The contract must hold sufficient NEST balance to cover all outstanding rNEST conversions (totalSupply of rNEST ≤ NEST balance held).
INV 2: Only the owner can mint rNEST tokens and recover NEST from the contract.
INV 3: Conversion ratio is fixed at 40% liquid NEST and 60% veNFT with 182-day lock duration. These values are immutable constants.
INV 4: Zero amount conversions are rejected.

Issue_30	RNest recover function can make rNEST unusable
Severity	**Governance**
Description	The recoverToken function in RNest allows the owner to withdraw NEST tokens from the contract. However, the rNEST contract requires NEST tokens to be available for users to convert their rNEST to NEST or veNEST. If the owner recovers too many NEST tokens, users holding rNEST will be unable to convert to NEST, the convertToVeNest and convertToNest functions will fail due to insufficient NEST balance, and rNEST holders are effectively left with worthless tokens.
Recommendations	Add validation to prevent withdrawing below the required backing amount (NEST balance >= rNEST totalSupply).
Comments / Resolution	Acknowledged.

TokenPublicRaiseUpgradeable

The TokenPublicRaiseUpgradeable contract implements a fixed-rate public sale that accepts native currency (ETH) and accounts purchased token amounts per depositor. Unlike NestRaiseUpgradeable, this contract does not transfer sale tokens directly. It only tracks allocations on-chain via userTokensAllocated for off-chain or separate distribution.

Users deposit native currency during the active raise window by calling deposit or simply sending ETH to the contract via receive. The contract calculates token allocations based on tokenPricePerOneNative (tokens per 1e18 native units). Deposits must meet the minimum amount threshold (when combined with previous deposits) and cannot exceed the per-user maximum or global cap. The contract enforces both a per-user cap via maxDepositAmount and a global cap via totalDepositCap. Deposits are only accepted when the current timestamp falls within the configured raise window (startTimestamp to endTimestamp inclusive). After the raise ends, the owner can withdraw all collected native currency to the treasury address via withdrawToTreasury.

Privileged Functions

- setTokenPricePerOneNative
- setTreasury
- setDepositLimits
- setRaiseWindow
- withdrawToTreasury

Core Invariants:

INV 1: startTimestamp < endTimestamp for a valid raise window.
INV 2: minDepositAmount <= maxDepositAmount for valid deposit limits.
INV 3: User's total deposited amount cannot exceed maxDepositAmount.
INV 4: Total deposits across all users cannot exceed totalDepositCap.
INV 5: Deposits are only accepted when block.timestamp is within [startTimestamp, endTimestamp].
INV 6: Treasury withdrawal is only allowed after the endTimestamp has passed.

Issue_31	Missing slippage protection in deposit function
Severity	Low
Description	The deposit function calculates tokensOut based on tokenPricePerOneNative, which can be changed by the owner at any time via setTokenPricePerOneNative. This creates a front-running risk where the owner could change the price right before a user's transaction is executed, resulting in the user receiving fewer tokens than expected. The deposit function has no slippage protection mechanism, meaning users cannot specify a minimum amount of tokens they expect to receive. This allows the owner to change the exchange rate to the user's disadvantage, especially in scenarios where the price is changed between transaction submission and execution.
Recommendations	Consider adding a minTokensOut parameter to the deposit or document and acknowledge this behaviour.
Comments / Resolution	Acknowledged.

Issue_32	Attacker can grief deposits by frontrunning
Severity	**Low**
Description	The _deposit function in TokenPublicRaiseUpgradeable calculates the remaining global capacity and reverts if the user's msg.value exceeds that amount. Because the check amount > maxLimit is strict, even a 1 wei difference caused by a prior transaction will trigger a revert. Hence, an attacker can monitor the mempool for large deposits intended to hit the totalDepositCap. By injecting a minDepositAmount deposit first, the attacker shifts the totalDeposited state, making the victim's amount invalid.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_33	Redundant __Ownable_init call in initialize
Severity	**Informational**
Description	In the initialize function, __Ownable_init is called explicitly, but since the contract inherits from Ownable2StepUpgradeable and __Ownable2Step_init is also called, the __Ownable_init call is redundant. The __Ownable2Step_init function internally calls __Ownable_init, so calling it separately results in unnecessary gas costs.
Recommendations	Remove the redundant __Ownable_init call from the initialize function, as __Ownable2Step_init already initializes the Ownable functionality.
Comments / Resolution	Acknowledged.

VeArtProxy

The VeArtProxy contract generates on-chain SVG artwork for veNFT tokens. It is called by VotingEscrow when tokenURI is requested and returns a base64-encoded JSON metadata structure containing an embedded SVG image.

The contract fetches the token's state from VotingEscrow including balance, voting power, lock end timestamp, permanent lock status, and attachment status. For attached NFTs, it queries the ManagedNFTManager to get the managed token ID, then retrieves balance and strategy name from the strategy contract. The SVG is assembled by combining static template parts from VeArtProxyStatic with dynamic data such as formatted balance, voting power, lock end date, and token ID. All values are formatted for display using the NumberFormatter and DateTime libraries.

Core Invariants:

INV 1: Contract is stateless and read-only. All data is fetched from external contracts (VotingEscrow, ManagedNFTManager, strategy contracts).

Issue_34	Redundant storage read in generateSVG
Severity	**Informational**
Description	In the generateSVG function, artStatic is already defined and cached from artProxyStatic, but the function reads artProxyStatic from storage again when calling IVeArtProxyStatic(artProxyStatic).startPart instead of using the cached artStatic variable.
Recommendations	Use the cached artStatic.startPart instead of IVeArtProxyStatic(artProxyStatic).startPart.
Comments / Resolution	Acknowledged.

VeArtProxyStatic

The VeArtProxyStatic contract stores static SVG template fragments used by VeArtProxy to generate veNFT artwork. It holds two string parts: startPart containing the opening SVG elements and styling, and endPart containing the closing SVG elements.

Each template part can only be set once by anyone via setStartPart and setEndPart. Once set, the setupStart and setupEnd booleans are flipped to true, preventing any future modifications. This ensures the artwork templates remain immutable after initial configuration.

Core Invariants:

INV 1: startPart can only be set once (setupStart must be false).
INV 2: endPart can only be set once (setupEnd must be false).
INV 3: Once set, template parts are immutable for the lifetime of the contract.

No issues were identified in this contract.

VeBoostUpgradeable

The VeBoostUpgradeable contract provides boosting functionality for users who lock NEST tokens in VotingEscrow. Users who meet minimum USD value and lock duration thresholds receive additional NEST tokens added to their lock, increasing their voting power without requiring additional deposits from the user.

The boost amount is calculated as a configurable percentage (default 10%) of the deposited NEST amount. Eligibility requires the deposit to meet a minimum USD value threshold (converted via a price provider oracle) and the lock duration to meet a minimum time threshold (default ~6 months). When VotingEscrow processes a boosted lock, it calls beforeNESTBoostPaid which validates the boost amount and transfers the boost NEST from this contract's balance. Additionally, when a boost is paid, the contract proportionally distributes any reward tokens it holds to the boost recipient. The reward distribution is based on the ratio of the paid boost amount to the contract's total NEST balance, ensuring users who receive larger boosts also receive proportionally more reward tokens.

Privileged Functions

- setPriceProvider
- setNESTBoostPercentage
- setMinUSDAmount
- setMinLockedTime
- addRewardToken

- removeRewardToken
- recoverTokens

Core Invariants:

INV 1: Paid boost amount cannot exceed the calculated eligible boost based on deposited amount and boost percentage.
INV 2: Paid boost amount cannot exceed the available NEST balance and allowance held by the contract.
INV 3: Minimum lock time cannot exceed MAXTIME (182 days).
INV 4: NEST token cannot be added as a reward token.
INV 5: Only VotingEscrow can call beforeNESTBoostPaid.

Issue_35	Multiple typos in naming conventions throughout the VeBoost contract
Severity	**Informational**
Description	The VeBoostUpgradeable contract contains several typos in naming conventions that should be corrected for consistency and clarity: 1. "calcualtions" instead of "calculations" 2. "_NEST_PREICSION" instead of "_NEST_PRECISION" 3. "_MAXTIME" instead of "_MAX_TIME" 4. "Stora" instead of "Store": 5. "nestBoostToBalanceRation" instead of "nestBoostToBalanceRatio"
Recommendations	Correct all typos to use proper spelling and consistent naming conventions.
Comments / Resolution	Acknowledged.

Issue_36	Redundant allowance check in getAvailableBoostNESTAmount
Severity	**Informational**
Description	In getAvailableBoostNESTAmount, the function checks the allowance between the contract and votingEscrow and compares it with the balance. However, in initialize, unlimited approval type(uint256).max is given to votingEscrow, meaning the allowance will always be the maximum value and can never be used up during the contract's lifetime. As a result, the allowance check is redundant and the function will always return availableBalance, making the extra storage read and comparison unnecessary.
Recommendations	Either remove the allowance check or acknowledge this issue.
Comments / Resolution	Acknowledged.

VeNestDistributorUpgradeable

The VeNestDistributorUpgradeable contract handles batch veNFT airdrops by locking NEST tokens in VotingEscrow on behalf of recipients. It provides a controlled mechanism for distributing veNFT positions to multiple users in a single transaction with configurable lock parameters per recipient.

Each distribution requires a whitelisted reason string for tracking and auditing purposes. The admin can whitelist or unwhitelist reasons via setWhitelistReasons. When distributeVeNest is called, the contract first validates that the reason is whitelisted and that the contract holds sufficient NEST balance to cover all distributions. It then iterates through the AirdropRow array and calls VotingEscrow.createLockFor for each recipient. Each row specifies the recipient address, NEST amount, lock duration, whether to enable permanent lock, and an optional managed token ID for attachment. The contract emits individual AirdropVeNest events for each recipient and a summary AidropVeNestTotal event for the batch.

Privileged Functions

- setWhitelistReasons
- distributeVeNest
- recoverTokens

Core Invariants:

INV 1: Distribution reason must be whitelisted before any distribution can occur.
INV 2: Recipient address cannot be zero address in any airdrop row.
INV 3: Contract must hold sufficient NEST balance to cover the total distribution amount before executing.
INV 4: Reason and isWhitelisted arrays must have matching lengths when setting whitelist.

Issue_37	Multiple typos in naming conventions throughout the VeNestDistributor contract
Severity	**Informational**
Description	The VeNestDistributorUpgradeable contract and its interface contain several typos in naming conventions that should be corrected for consistency and clarity: 1. "Aidrop" instead of "Airdrop" 2. "arrayies" instead of "arrays"
Recommendations	Correct all typos to use proper spelling.
Comments / Resolution	Acknowledged.

VeNestSplitMerkIAidropUpgradeable

The VeNestSplitMerkIAidropUpgradeable contract implements a Merkle-based airdrop where eligible users can choose between receiving discounted liquid tokens or full-value veNFT locks. User eligibility and allocation amounts are determined by a Merkle tree, and claims are verified via Merkle proofs.

Users have two claiming options. If they choose inPureTokens as true, they receive liquid NEST tokens at a discounted rate defined by pureTokensRate (e.g., 0.8e18 means 80% of their allocation). If they choose inPureTokens as false, they receive the full allocation amount as a veNFT locked for 182 days via VotingEscrow.createLockFor. When claiming as veNFT, users can optionally enable permanent lock or attach to a managed token ID. The contract tracks cumulative claims per user via userClaimed, allowing partial claims over time up to the total allocation in the Merkle tree. Authorized claim operators can call claimFor to process claims on behalf of users, enabling batch processing. The contract is pausable, and configuration changes such as setting the Merkle root or pure tokens rate can only be made while paused.

Privileged Functions

- pause
- unpause
- setMerkIroot
- setPureTokensRate
- setIsAllowedClaimOperator
- recoverToken

Core Invariants:

INV 1: pureTokensRate cannot exceed 1e18 (100%).
INV 2: User's cumulative claimed amount cannot exceed their allocation in the Merkle proof.
INV 3: Merkle root and pure tokens rate can only be modified when the contract is paused.
INV 4: Only the user themselves or an allowed claim operator can call claimFor for a target address.
INV 5: Claims revert with ZeroPureTokensRate if claiming in pure tokens when pureTokensRate is zero.

Issue_38	Airdrop operator can force claim with undesirable settings
Severity	**Governance**
Description	The claimFor function in VeNestSplitMerkIAidropUpgradeable allows an allowed operator to claim on behalf of any user with specific lock settings. This means an operator can force a user into a claim configuration they didn't choose, such as permanent locks when they wanted liquid tokens, or attaching their position to a managed NFT without consent. A malicious or careless operator could front-run user's own claim transaction with undesirable parameters, effectively trapping user funds in configurations they can't easily exit.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_39	Multiple typos in naming conventions throughout the VeNestSplitMerkIAidrop contract
Severity	**Informational**
Description	The VeNestSplitMerkIAidropUpgradeable contract and its interface contain typos in naming conventions that should be corrected for consistency and clarity: "Aidrop" instead of "Airdrop"
Recommendations	Correct all typos to use proper spelling.
Comments / Resolution	Acknowledged.

VoterUpgradeableV2

The VoterUpgradeableV2 contract works alongside the VotingEscrowUpgradeableV2 contract but has a very distinct role. The contract is in charge of veNFT voting and emissions distribution. It integrates with VotingEscrowUpgradeableV2, MinterUpgradeable, Gauges, v2/v3 pools, bribes, and strategies. Users vote to direct emission to gauges, and voting power is allocated across multiple pools. The voter contract also has special handling for managed NFTs, and a whitelist system that allows for instant voting.

Appendix: Whitelist System

Whitelisted NFTs are allowed to bypass the vote delay restrictions. This allows for immediate re voting without waiting for the vote delay. This is critical for managed NFTs and automated strategies. The isWhitelistedNFT mapping tracks the privileged NFTs and Governance is allowed to add or remove NFTs from the whitelist.

Appendix: Multi Pool Voting

The vote function accepts arrays of pools and respective weights. Users can distribute voting power across multiple gauges simultaneously, Weight validation ensures that the sum does not exceed the users total voting power. Pools are tracked via poolVote mapping.

Appendix: V2 and V3 Pool Support

V2 and V3 pools have separate factory contracts and their own gauge types(V2PairsGauge and V3PairsGauge). Both pool types participate in the same voting/distribution system. The factory whitelist is required for gauge creation.

Privileged Functions

- updateAddress
- setVotingPause
- setDistributionWindowDuration
- killGauge
- reviveGauge
- createV2Gauge
- createV3Gauge
- createCustomGauge

Core Invariants:

INV 1: Only the Minter contract can call notifyRewardAmount to inject emissions into the voting system.
INV 2: A pool can have at most one gauge. Attempting to create a gauge for a pool that already has one reverts.
INV 3: Gauges can only be created for pools deployed by the registered V2 or V3 pool factories, except for custom gauges which bypass factory validation.
INV 4: Voting actions are restricted to the voting window, which excludes the distributionWindowDuration at the start and end of each epoch week.
INV 5: Only the veNFT owner or approved operator can vote, reset, poke, or attach/detach on behalf of a tokenId.
INV 6: When a gauge is killed, any pending claimable rewards are returned to the Minter. Killed gauges do not receive new emissions but votes for them are still subtracted from the voter on reset.
INV 7: The sum of weightsPerEpoch across all pools for a given epoch should equal totalWeightsPerEpoch for that epoch.
INV 8: Reward distribution to a gauge is proportional to its pool's voting weight relative to the total weight, calculated as poolWeight times the delta in global index.

Issue_40	Reward index calculation is diluted by killed gauges
Severity	**Medium**
Description	Reward distribution for all users will be lower than intended because _reset fails to subtract the voting power of killed gauges from the global totalWeightsPerEpoch. A user's voting weight is added to both the pool-specific weightsPerEpoch and the global totalWeightsPerEpoch during the vote or poke process. When a gauge is killed via killGauge, its isAlive status is set to false. If a user who previously voted for that gauge subsequently calls reset, vote, or poke, _reset does not increment totalVotePowerForPools by the voting power of the killed gauge which means that the global totalWeightsPerEpoch is never decreased by the user's old weight. This leads to a "Ghost Weight" inflation in the denominator during the notifyRewardAmount calculation: *uint256 weightAt = totalWeightsPerEpoch[epochTimestamp() - _WEEK];* *if (weightAt > 0) {* * index += (amount_ * 1e18) / weightAt;* *}* Since weightAt is artificially high, the index (reward-per-unit-of-weight) increases by a smaller amount than it should. Consequently, every active gauge in the system receives fewer rewards.
Recommendations	Consider decrementing the totalWeightsPerEpoch by the weightsPerEpoch and then setting the weightsPerEpoch to 0 when killing a gauge. This also requires changes to _reset such that the weightsPerEpoch is not deducted by the votePowerForPool for inactive gauges.
Comments / Resolution	Fixed following recommendation.

Issue_41	Revert due to underflow for gauges killed and revived in same epoch
Severity	**Low**
Description	When a gauge is killed, its weightsPerEpoch is set to 0. But when it is revived, the weightsPerEpoch value is not reset and stays 0. This leads to an edge case if a gauge is killed and then revived in the same epoch. weightsPerEpoch is set to 0 when killed, and on revive it stays at 0. When _reset is triggered by a user, weightsPerEpoch tries to deduct the voting power of that pool again, since the gauge is alive *if (gaugesState[gauge].isAlive) {* *weightsPerEpoch[time][pool] -= votePowerForPool;* *totalVotePowerForPools += votePowerForPool;* *}* This causes a revert due to underflow, since the weightsPerEpoch value is 0.
Recommendations	Consider preventing revival of killed gauges in the same epoch, or resetting their voting state to prevent the underflow.
Comments / Resolution	Acknowledged.

Issue_42	Rewards notified to the Voter contract can be lost
Severity	**Medium**

Description	When rewards are notified to the Voter contract the index is updated proportionally to the voting weight of the previous epoch:
	
	function notifyRewardAmount(uint256 amount_) external {
	_checkSender(minter);
	
	IERC20Upgradeable(token).safeTransferFrom(_msgSender(), address(this), amount_);
	uint256 weightAt = totalWeightsPerEpoch[epochTimestamp() - _WEEK]; // minter call notify after updates active_period, loads votes - 1 week
	if (weightAt > 0) {
	*index += (amount_ * 1e18) / weightAt;*
	}
	emit NotifyReward(_msgSender(), token, amount_);
	}
	
	The problem arises when the totalWeightsPerEpoch[] for the previous epoch was 0 i.e. no user voted in that epoch and in that case the notified token amount is not accounted for and is stuck in the voter contract forever.

Recommendations	In the case where totalWeightsPerEpoch is 0, transfer the notified amount back to the Minter contract.
Comments / Resolution	Fixed, a transfer is only initiated for non zero values of totalWeightsPerEpoch.

Issue_43	Cumulative index distribution mechanism breaks when gauge is not distributed in an epoch
Severity	**Medium**
Description	The current distribution mechanism in _distribute uses a cumulative index approach that assumes every gauge is distributed every epoch. When notifyRewardAmount is called, it updates the global index based on the previous epoch's total weights. However, _distribute calculates rewards using delta = index - state.index and multiplies it by only the last epoch's weight: weightsPerEpoch[currentTimestamp - _WEEK][state.pool]. This creates a critical flaw: if a gauge is not distributed in epoch N (but the global index was updated), and then distributed in epoch N+1, the calculation will be incorrect. The delta will represent 2 epochs worth of index increase, but totalVotesWeight will only reflect epoch N+1's weight. If epoch N had low votes and epoch N+1 had high votes, the gauge will receive incorrect rewards - it will get rewards calculated for 2 epochs but only using the weight from 1 epoch. The current system heavily depends on the assumption that distribute will be called for every gauge every epoch, which is not guaranteed (only incentivized). This edge case, while rare, can lead to incorrect reward distribution.
Recommendations	Document this behaviour and acknowledge this issue. Epoch distributions must not be missed.
Comments / Resolution	Partially fixed. For 0 weight epochs, the for loop in _distribute does not update state.index, leading to the same issue.

Issue_44	Incorrect accounting in _reset when gauge is killed between vote and reset
Severity	**Medium**
Description	In the _reset function, there is a check *if (gaugesState[gauge].isAlive)* before adding votePowerForPool to totalVotePowerForPools. This creates an accounting issue when a gauge is killed between when a user votes and when they reset their votes. When a user votes via _vote, totalWeightsPerEpoch[time] is always incremented regardless of gauge state (since voting for a killed gauge reverts). However, when resetting votes, if the gauge was killed in the meantime, the isAlive check prevents totalVotePowerForPools from including that vote power. This means totalWeightsPerEpoch[time] is not properly decremented, leaving inflated weights in the epoch. Since totalWeightsPerEpoch is used in notifyRewardAmount to calculate the global index for reward distribution, this inflation dilutes rewards for all gauges in that epoch. The vote power should always be deducted from totalWeightsPerEpoch since it was added when voting, regardless of the gauge's current state.
Recommendations	For dead gauges, their voting power must be also removed in the reset function. Otherwise, document the behaviour and acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_45	epochTimestamp uses stale active_period which can cause votes to be added to wrong epoch
Severity	**Low**
Description	The epochTimestamp function uses IMinter(minter).active_period, which returns a stored state variable that is only updated when update_period is called in the Minter contract. This creates a critical issue: if update_period hasn't been called yet in a new epoch (e.g., due to chain downtime or synchronizer issues during epoch switch), active_period will still return the previous epoch's timestamp. When _vote is called, it uses epochTimestamp to determine which epoch to add votes to. Even though _checkStartVoteWindow allows voting only after 1 hour from epoch start (to give time for update_period to be called), if the chain was non-operational during the epoch switch and update_period still hasn't been called, votes will be incorrectly added to the old epoch. This dilutes rewards in the old epoch and creates incorrect accounting. The Minter contract has a period function that always calculates the current epoch correctly using (block.timestamp / WEEK) * WEEK, independent of whether update_period has been called. Using this function would ensure votes are always added to the correct epoch, and would also allow removing the vote window checks since the epoch calculation would always be accurate.
Recommendations	Change epochTimestamp to use IMinter(minter).period instead of IMinter(minter).active_period. Or during voting, either invoke an update_period call if the epoch needs to be changed, or revert until the epoch is changed in the Minter contract.
Comments / Resolution	Acknowledged.

Issue_46	_updateLastVotedTimestamp should use block.timestamp instead of epochTimestamp + 1
Severity	**Medium**
Description	The _updateLastVotedTimestamp function sets lastVotedTimestamps[tokenId_] to epochTimestamp + 1, which means all votes cast during the same epoch get the same timestamp (epoch start + 1 second). This doesn't accurately track when the vote actually occurred. The lastVotedTimestamps is used in _checkVoteDelay to compare against block.timestamp for vote delay enforcement. If multiple votes are cast in the same epoch, they would all have the same timestamp, making the delay check ineffective. Additionally, the timestamp should reflect the actual time when the vote was cast, not an approximation based on the epoch. If the voting cooldown is less than 1 week, this also allows veNFTs to vote multiple times violating the cooldown period.
Recommendations	Change _updateLastVotedTimestamp to use block.timestamp instead of epochTimestamp + 1 to accurately track when votes are cast. This ensures proper functionality of vote delay checks and provides accurate timestamp tracking. This issue breaks the vote delay restriction, but since the current contract has no way of setting the voteDelay value, this does not pose an immediate threat and the vote delay mechanism as a whole is currently defunct.
Comments / Resolution	Acknowledged.

Issue_47	lastDistributionTimestamp not updated for killed gauges with zero claimable
Severity	**Informational**
Description	In the _distribute function, when a killed gauge is distributed, the index is always updated and any rewards are returned to the minter contract. However, lastDistributionTimestamp is only updated when claimable > 0 && state.isAlive, meaning it won't be updated for killed gauges with zero claimable amount. This creates an inconsistency where the index reflects that distribution occurred, but lastDistributionTimestamp does not, even though the rewards designed for that gauge were returned to the minter. The state should always be updated consistently, regardless of whether there is something to claim.
Recommendations	Update lastDistributionTimestamp whenever distribution logic runs, not only when claimable > 0 && state.isAlive. Move the lastDistributionTimestamp update outside the conditional block to ensure it's always updated when the index is updated, maintaining state consistency.
Comments / Resolution	Acknowledged.

Issue_48	_poke fails for managed NFTs when votes include killed gauges, blocking attach/detach operations
Severity	**Low**
Description	When a managed NFT has votes cast on gauges that are later killed, the _poke function becomes unusable because it calls _vote, which reverts if any gauge in the vote is killed. This creates an issue for users trying to attach or detach from managed NFTs. The problem occurs in several scenarios: - attachToManagedNFT calls _poke(managedTokenId_) to update the managed token's votes - dettachFromManagedNFT calls _poke(managedTokenId_) if the managed token still has weight - onDepositToManagedNFT also calls _poke(managedTokenId_) when deposits occur For normal users, this is acceptable because they can call vote with new pools that exclude killed gauges. However, for managed NFTs, _poke is called automatically during attach/detach operations and cannot be modified by the user. If the managed token has votes on any killed gauge, these operations will fail, effectively locking users out of managing their positions.
Recommendations	Consider documenting this behaviour and acknowledging this issue. When gauges are killed, neVFTs voting for it must be reset first.
Comments / Resolution	Acknowledged.

Issue_49	Unused voteDelay variable should be marked as deprecated
Severity	Informational
Description	The voteDelay variable in VoterUpgradeableV2 is declared but never initialized, defaults to 0, and has no setter function to change its value. Since voteDelay is always 0, the _checkVoteDelay function is effectively useless and adds unnecessary gas costs to multiple operations without providing any meaningful protection.
Recommendations	Consider removing the dead code or acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_50	onAfterTokenMerge does not correctly reset state in case user A merges to user B's token
Severity	Informational
Description	onAfterTokenMerge is called on voter after a token is merged, which resets the target of calling users' compound emissions. In case a token is merged from user A to user B, user As state is not reset. This leaves user As compound configuration corrupted, but does not lead to any issues, as tokens are burned when they are merged.
Recommendations	Remove the merged token from target locks in case user A merges to user B.
Comments / Resolution	Acknowledged.

Issue_51	votingHook is not invoked under an edge case
Severity	Informational
Description	In detachFromManagedNFT in Voter if weight after detaching is 0 then _reset is performed but the vote flag is not switched i.e. votingHook is not called to update it to false similar to how we do in reset.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_52	Index for a gauge is not updated upon revival
Severity	Informational
Description	When a gauge is revived the index value is not reset to the current value of the global index, therefore if distribute was never called on the killed gauge for the epochs it was killed for, upon revival it will accrue rewards for all those epochs.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

VotingEscrowUpgradeableV2

The VotingEscrowUpgradeable is a voting escrow contract that allows users to lock/deposit tokens in order to receive a corresponding veNFT token which includes the amount locked plus an additional optional bonus if requirements are met. This contract extends the regular functionality and adds several new features such as permanent locks, a managed attachment system, boost system integration, upgradeability and much more. All while maintaining the key components of the regular voting escrow contracts such as linear voting power decay, checkpoint system for historical voting power, week based epoch system, Lock amounts and duration mechanics, and supply tracking that can be queried via timestamps.

Appendix: Permanent Locking System

A major component of the VotingEscrowUpgradeableV2 contract is the permanent locking system. Permanent locking allows a veNFT to maintain the same consistent voting power without any decay. Permanent locks remain indefinitely and have their own separate permanent supply tracking via permanentTotalSupply. Users can opt out of permanent locks at any time but once unlocked they will start a non permanent lock at the max lock time.

Appendix: Managed NFTs and Attachment System

The managed NFTs and Attachment System is deeply integrated into the VotingEscrowUpgradeableV2 contract. Managed NFTs are a special administrative type of veNFT that allows holders of the regular veNFT type to attach to them. When a user attaches to a managed NFT, they transfer voting power and control but retain ownership of their NFT and can detach as they wish. The ManagedNFTManager is the main orchestrator behind this system, and managed NFTs are a critical component of auto compounding strategies and protocol owned liquidity.

Appendix: Boost Integration (VeBoost)

Users who lock their tokens beyond a certain time and amount threshold may be eligible for a boost. The VeBoost contract provides tokens for eligible users, allowing admins to configure the amount of boost each NEST token qualifies for. A user can opt in to boosts by setting the shouldBoosted boolean to true when creating a lock. Boosts ultimately incentivize longer lock durations in return for more voting power without requiring the user to deposit more tokens.

Appendix: NFT Based Lock Model

Each lock is its own separate NFT token that can be freely transferred. Locks are not bound to accounts, thus users may own several different locks. Locks are tradable when not attached or voting, this allows users to exchange voting power on a secondary market.

Appendix: Bribe Burning Mechanism

The bribe burning mechanism exists in the burnToBribes function and allows for early exit on locks. burnToBribes will burn the veNFT and return the underlying token, which is integrated with the customBribeRewardReceiver. The underlying tokens are sent to the reward receiver and users are minted brVeNEST tokens.

Appendix: Voter Contract Integration

The VotingEscrowUpgradeableV2 contract is also deeply integrated with the VoterUpgradeableV2 contract. Before and after transfers, the contract will call into the voter in order to reset votes if voted, and to change emission targets for the veNFT. The voter also handles transferring voting power when attaching and detaching from a managed NFT.

Privileged Functions

- updateAddress
- createManagedNFT
- votingHook
- onAttachToManagedNFT
- burnToBribes

Core Invariants:

INV 1: The supply variable must equal the sum of all locked amounts across all veNFTs.
INV 2: The permanentTotalSupply must equal the sum of locked amounts for all permanently locked veNFTs plus all attached veNFT balances held in managed NFTs.
INV 3: Voting power for non-permanent locks decays linearly from the locked amount to zero over the lock duration. The formula is bias minus slope multiplied by time elapsed.
INV 4: Permanent locks have constant voting power equal to their locked amount with no decay.
INV 5: Only the owner or approved operator of a veNFT can withdraw, merge, lock permanent, unlock permanent, increase unlock time, or burn to bribes.
INV 6: Tokens cannot be withdrawn from a veNFT until its lock has expired, unless it is permanently locked and first unlocked.
INV 7: Attached veNFTs cannot be withdrawn or have their lock time modified directly. They must be detached first.
INV 8: The balanceOfNFT function returns zero in the same block as a transfer to prevent flash loan manipulation of voting power.
INV 9: When a veNFT is attached to a managed NFT, its locked balance is added to the managed NFT and its own locked amount is set to zero while the isAttached flag is set to true.

Issue_53	Merge reads zeroed lock end time leading to incorrect voting weight
Severity	**High**
Description	When merging two veNFT tokens, the merge function is supposed to use the longer lock duration between the two tokens. However, due to incorrect ordering of operations, the lock end time of the source token is always read as zero, causing the merged token to potentially use a shorter lock duration than intended. In the merge function, the _withdrawClearNftInfo is called first which resets the tokenFromId lock data to zero: *uint256 mergeAmount = _withdrawClearNftInfo(tokenFromId_);* *uint256 fromLockedEnd = nftStates[tokenFromId_].locked.end;* The _withdrawClearNftInfo function internally calls _updateNftLocked with a zeroed LockedBalance struct: *_updateNftLocked(tokenId_, LockedBalance(0, 0, false));* This sets the locked.end value to 0 before the fromLockedEnd variable is assigned. Subsequently when _proccessLockChange is called, the ternary comparison uses this zeroed value: *_proccessLockChange(* *tokenToId_,* *mergeAmount,* *fromLockedEnd >= stateTo.end ? fromLockedEnd : stateTo.end,* *stateTo,* *DepositType.MERGE_TYPE,* *false* *);* Since fromLockedEnd is always 0 at this point, the comparison will always evaluate to false and stateTo.end will be used regardless of what the original lock end time of tokenFromId was.

	This breaks the intended behavior where merging should preserve the longer lock duration. For example if tokenFromId has a 4 year lock and tokenToId has a 1 year lock, the merged token will incorrectly have only a 1 year lock instead of 4 years. Since voting power in the ve model is calculated based on lock duration, this results in significantly reduced voting weight for the merged position and an incorrect lock duration.
Recommendations	Cache the fromLockedEnd value before calling _withdrawClearNftInfo to preserve the original lock end time.
Comments / Resolution	Fixed by following recommendations.

Issue_54	Checkpoint function fails to reset negative bias and slope values
Severity	Low
Description	The _checkpoint function contains conditional checks for negative bias and slope values but fails to actually reset them to zero. This creates inconsistent behavior when compared to votingPowerTotalSupply which properly handles this case. In _checkpoint, the function checks if last_point.bias or last_point.slope are negative but the statements inside the conditionals do nothing: *if (last_point.bias < 0) {* *// This can happen* *last_point.bias;* *}* *if (last_point.slope < 0) {* *// This cannot happen - just in case* *last_point.slope;* *}* The lines last_point.bias and last_point.slope are simple variable references that have no effect. They do not assign the value to zero. In contrast, the votingPowerTotalSupply function properly handles the negative bias case: *if (last_point.bias < 0) {* *last_point.bias = 0;* *}* This inconsistency means that negative bias and slope values can be stored in supplyPointsHistory during checkpoints. Since these checkpoint values are used to calculate voting power across the protocol, negative values being stored can corrupt the global supply tracking and lead to incorrect voting power calculations for all

	users.
Recommendations	Add the assignment to zero inside each conditional block in _checkpoint to match the behavior of votingPowerTotalSupply:
Comments / Resolution	Fixed by following recommendation.

Issue_55	compoundEmissionClaim can be temporarily DoSed
Severity	Medium
Description	compoundEmissionClaim can be temporarily DoSed due to an incorrect state updation logic on merge , consider the following scenario: 1.) There are 2 users Alice and Bob , Alice has approved Bob for all her tokens. 2.) In the CompoundEmissionExtension the _usersCompoundEmissionTargetLocks for Alice is as follows [1,2,3,4] where 1,2,3,4 are tokenIds. 3.) Bob also owns his own tokenIds and his _usersCompoundEmissionTargetLocks is [5,6,7,8] 4.) Since Bob has approval for Alice's tokens, he calls merge where fromTokenId is 7 and toTokenId is 3 5.) The logic burns the tokenId 7 and tokenId 3's locked amount is incremented. 6.) In onAfterMerge we update Alice's _usersCompoundEmissionTargetLocks since she is the owner of the toTokenId , therefore Alice's new _usersCompoundEmissionTargetLocks is [1,2,3,4] (same because she doesn't have tokenId 7) 7.) Now the issue is that _usersCompoundEmissionTargetLocks for Bob is still [5,6,8,7] which is incorrect because tokenId 7 doesn't exist now. This happens because when _burn is performed and _afterTokenTransfer hook is invoked it does not enter the if branch since it is a burn op. Now whenever compoundEmissionClaim is done for Bob it will revert since it will try to perform a depositFor for a tokenId that doesn't exist.
Recommendations	The afterMerge logic should also update Bob's _usersCompoundEmissionTargetLocks so that it tracks the correct tokenIds.
Comments / Resolution	Fixed by only allowing merging of NFT if the owner owns both tokens.

Issue_56	Deposits to attached tokens do not receive boost rewards
Severity	**Informational**
Description	The depositFor function handles deposits differently based on whether a token is attached to a managed NFT. When a token is attached, the function routes to _depositToAttachedNFT instead of _depositFor. The issue is that _depositToAttachedNFT contains no boost logic whatsoever, while the standard _depositFor path grants boost rewards through _proccessLockChange. In depositFor, the routing occurs as follows: *function depositFor(uint256 tokenId_, uint256 amount_, bool shouldBoosted_, bool withPermanentLock_) external override nonReentrant {* *if (nftStates[tokenId_].isAttached) {* *_depositToAttachedNFT(tokenId_, amount_);* *} else {* *_depositFor(tokenId_, amount_, shouldBoosted_, withPermanentLock_);* *}* *}* The _depositToAttachedNFT function simply transfers tokens and updates balances without any boost calculation: *function _depositToAttachedNFT(uint256 tokenId_, uint256 amount_) internal {* *IERC20Upgradeable(token).safeTransferFrom(_msgSender(), address(this), amount_);* *supply += amount_;* *permanentTotalSupply += amount_;* *// Updates managed token locked balance* *// No boost logic present* *}*

	Meanwhile _depositFor calls _proccessLockChange which contains the full boost mechanism. When shouldBoosted is true and the deposit meets minimum requirements, additional NEST tokens are pulled from the VeBoost contract and added to the locked balance. This creates inconsistent behavior within a single function. A user depositing to an attached token receives no boost, but could receive the boost by detaching their token, making the deposit, then reattaching. Both scenarios involve the same underlying action of adding funds to a position, yet one is rewarded and the other is not. This issue is marked as informational since this functionality will not be used by the protocol team.
Recommendations	If this is a design choice, consider acknowledging the issue. Otherwise consider adding boosts to deposits that deposit to an attached NFT.
Comments / Resolution	Acknowledged.

Issue_57	lastPoint.blk calculation will always be incorrect
Severity	**Low**
Description	The checkpoint logic uses the last point's block and calculates the block_slope: *block_slope = (1e18 * (block.number - last_point.blk)) / (block.timestamp - last_point.ts);* It also stores the initial_last_point which is used to update the the block number , but the initialization of the initial_last_point is incorrect: *Point memory last_point = Point({bias: 0, slope: 0, ts: block.timestamp, blk: block.number, permanent: 0});* *if (_epoch > 0) {* *last_point = supplyPointsHistory[_epoch];* *}* *uint256 last_checkpoint = last_point.ts;* *Point memory initial_last_point = last_point;* Point memory initial_last_point = last_point this line does not create an independent copy of last_point due to solidity's behavior , initial_last_point and last_point point to the same data in memory and any subsequent modifications to last_point are also reflected in initial_last_point , consider the following impact , in _checkpoint to calculate last_point.blk: *last_point.ts = t_i;* *last_point.blk = initial_last_point.blk + (block_slope * (t_i - initial_last_point.ts)) / 1e18;* But since both initial_last_point and last_point share the same memory location then (t_i - initial_last_point.ts) will be 0 because ti is last_point.ts and calculation of block delta will always be 0.
Recommendations	Consider changing the assignment to: *Point memory initial_last_point = Point(last_point.bias, last_point.slope, last_point.ts, last_point.blk);*
Comments / Resolution	Acknowledged.

Issue_58	Token transfer is done to the caller upon withdraw instead of the owner of the veNFT
Severity	**Informational**
Description	Upon withdraw in VotingEscrow the token amount is transferred to the _msgSender which can be the approved address too, whereas the token amount should be transferred to the owner of the veNFT.
Recommendations	Consider transferring the token amount to the owner of the veNFT else acknowledge the issue.
Comments / Resolution	Acknowledged.

Issue_59	Token sale can be frontrun to change tokens properties
Severity	**Informational**
Description	VotingEscrowUpgradeableV2 allows the transfer of NFTs between users, which allows them to trade them on external exchanges. In case a user wants to purchase a token, the sale can be frontrun to change the state of the NFTs, for example increasing the unlock time, changing the agreed sale assets.
Recommendations	Consider documenting this behaviour and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_60	Checkpoint states can be inflated with multiple transfers in the same block
Severity	**Low**
Description	In the _checkpoint function, the epoch value is updated on every invocation. *epoch = _epoch;* This basically causes a new checkpoint to be written for every single call. Thus the epoch can be incremented even if multiple transfers happen in the very same block. Other similar ve3,3 systems add a timestamp clause which checks for duplicate timestamps by checking _pointHistory[_epoch - 1].ts == block.timestamp. This also leads to the situation where multiple checkpoint values can exist at the same timestamp. This can cause issues when looking up historical values.
Recommendations	Consider overwriting the last checkpoint instead if _pointHistory[_epoch - 1].ts == block.timestamp.
Comments / Resolution	Acknowledged.

DexV2

The dexV2 folder contains the contracts necessary for a solidly-style CPAMM implementing both volatile and stableswap pools, plus the periphery needed to create pools, add/remove liquidity, and route swaps. Each deployed Pair contract maintains its own internal invariant, which is preserved during every swap.

Swap fees are physically separated from pool reserves and diverted into a separate contract PairFees, deployed 1:1 for each pool. Each pool also has an associated Gauge contract, which manages reward distribution for providing liquidity to that pool.

User-facing convenience is provided by the router layer, which creates pairs on demand (if allowed), calculates quotes, and performs multi-hop swaps across routes, while also handling WETH wrapping/unwrapping for ETH flows.

Pools can also be configured with volatile dynamic fees, where the fees start at some value and gradually step down over time to some other pre-defined value.

Pair

The Pair contract is the main contract that implements the CPAMM. During initialization, the _stable parameter passed in determines if the pool is going to be a volatile or a stable pool. In case of a volatile pool, the invariant $K = xy$ is preserved, while for stable pools, the invariant $K = x^3y + y^3x$ is preserved. Fee handling is split out: swap fees are removed from the pool and sent to a dedicated PairFees contract, and the pair maintains per-LP fee indexes (index0/index1 with per-user checkpoints) so LPs can later claim their accumulated fees using claimFees. The contract also maintains a lightweight TWAP oracle mechanism by storing cumulative reserve observations at most once per 30 minutes and offering sample/quote/prices to compute average prices from those observation deltas.

Privileged Functions

- setCommunityVault

Core Invariants:

INV 1: Swaps must respect the invariant function _k including fees.
INV 2: Swap must not allow outputs that exceed reserves, must require that at least one input was actually received, and must forbid sending output to token0 or token1 addresses.
INV 3: The initial mint permanently locks MINIMUM_LIQUIDITY to address(0), so once liquidity is created, totalSupply should never drop below that locked minimum.
INV 4: Any LP balance change must call _updateFor for both sender/recipient so that claimable0/claimable1 reflect the correct pro-rata share since the last checkpoint.
INV 5: For stable pools, the initial deposit must be with equal amounts of both tokens.

Issue_61	_get_y uses wrong function causing incorrect output estimates in getAmountOut
Severity	**Medium**
Description	In the _get_y function when handling convergence edge cases in the Newton-Raphson method, the code incorrectly uses _k instead of _f to check if y + 1 is the correct answer. This causes incorrect calculations in the view function getAmountOut, which is used by external contracts (like RouterV2) to estimate swap outputs. *if (_k(x0, y + 1) > xy) {* *return y + 1;* *}* The function _f takes 18-decimal normalized values as input while _k takes the raw amounts as input and then normalizes them. However, since x0 here is already normalized, this re-normalizes the amount resulting in an incorrect value due to the decimal increment. This breaks the convergence clause and gives incorrect inputs, leading to failed swaps for tokens which are not 18 decimals.
Recommendations	Replace _k with _f. *if (_f(x0, y + 1) > xy) { // Use _f() instead of _k()* *return y + 1;* *}* The corresponding check in the else branch correctly uses _f, which confirms that the if branch should also use _f.
Comments / Resolution	Fixed by following recommendation.

Issue_62	Stable pairs can be permanently broken
Severity	**Medium**
Description	The stable pair contract implements a stableswap like invariant $x^3y + y^3x >= k$ with fixed-point rescaling to 1e18 before computing k. The swap guard enforces monotonicity by reverting when _k[new] < _k[old]. Minting uses Uniswap-v2-style pro-rata math with reserves in the denominator. An initial-liquidity check enforces equalized deposits and a minimum k threshold only on the first mint. All arithmetic is integer-based with multiple divisions by 1e18, making the k calculation susceptible to truncation at low balances. An attacker can (i) bring reserves into a regime where k truncates to zero via depositing and withdrawing, (ii) perform swaps that would normally violate the invariant but now pass because the guard compares 0 to 0, and (iii) repeatedly mint at tiny reserves to inflate LP supply to overflow levels which then prevents further mints. The final outcome is a denial-of-service for the pair, and in VE-tied architectures this can functionally brick the affected token pair and related operations. **The sequence of actions is as follows:** Mint on the pool with amountX = 1e16 and amountY = 1e16. It mints 1e3 to reserve/dead address • It mints 9999999999999000 to attacker • Please note that at this step, k > 0 Burn 9999999999999000 from attacker Swap 1 tokenY in and receive all tokenX - 1 out • This works because K is zero and we can simply request any

	output amount Now there's totalSupply = 1e3, reserveX = 1, reserveY = 1. Attacker mints with 1e8 amountX and 1e8 amountY. • liquidityToMint = _amount0 * _totalSupply / _reserve0 • liquidityToMint = 1e8 * 1e3 / 1 • liquidityToMint = 1e11 Now we repeat the swap step and transfer out all reserves but leave 1 wei leftover to be able to mint again in the next step.
Recommendations	Consider ensuring that liquidity removal cannot result in k below MINIMUM_K.
Comments / Resolution	Partially fixed. The MINIMUM_K check is applied in the burn function for all pools, but it should only be applied for the stable pools. The check should be included in an if(stable) check as well.

Issue_63	sync allows updating reserves when pool has no liquidity, corrupting cumulative price data and observations
Severity	**Low**
Description	The sync function in Pair does not check if the pool has any liquidity (totalSupply == 0) before updating reserves. When called on an empty pool, it updates reserve state and creates observations that don't represent actual trading activity, which can corrupt cumulative price data and lead to incorrect TWAP calculations for downstream systems. *function sync() external lock {* *_update(IERC20(token0).balanceOf(address(this)), IERC20(token1).balanceOf(address(this)), reserve0, reserve1);* *}* When _update is called with zero liquidity, it still creates observations and updates cumulative prices based on reserves that don't represent a valid AMM state. This pollutes the observation array with invalid entries that can cause incorrect TWAP calculations when queried by oracle systems or other DeFi protocols.
Recommendations	Check the totalSupply before calling update in the sync function and revert if 0.
Comments / Resolution	Acknowledged.

Issue_64	Swap fees can be bypassed if any of the token is a low decimal token
Severity	**Informational**
Description	If any of the pool token is a low decimal token then the swap fees can be bypassed. For example, if the token is OUSD which has 2 decimals then an attacker can keep swapping with 0 fees since in *_update0((amount0In * factoryCache.getFee(address(this), stable)) / 10000);* it will round to 0 , even if the amountIn is 2000 OUSD i.e. 20 USD and fee is 4 (stable fee). This is because the fee cannot be represented in 2 decimals for small swap amounts and will thus round down to 0.
Recommendations	Consider acknowledging the issue or don't allow such low decimal tokens.
Comments / Resolution	Acknowledged.

PairFactoryUpgradeable

The PairFactoryUpgradeable contract is the central factory and configuration authority for dexV2 pairs. It's an upgradeable AccessControlUpgradeable contract that (a) deterministically deploys new Pair instances as minimal-proxy clones of a configurable implementation, (b) maintains the canonical registry mapping of (tokenA, tokenB, stable) -> pair (including reverse-direction lookups), and (c) governs protocol parameters such as default stable/volatile swap fees, protocol-fee share, and per-pair overrides.

Privileged Functions

- upgradePairImplementation
- setPause
- setCommunityVaultFactory
- setIsPublicPoolCreationMode
- setProtocolFee
- setCustomProtocolFee
- setCustomVolatileDynamicFeeModule
- setCustomFee
- setFee

Core Invariants:

INV 1: Only a single pair contract can be deployed for any (token0, token1, stable) combination.

Issue_65	Multiple typos in PairFactoryUpgradeable contract and interface
Severity	**Informational**
Description	The PairFactoryUpgradeable contract and IPairFactory interface contain several typos: 1. Error name: IncorrcectFee should be IncorrectFee - declared in interface and used multiple times 2. Comment: "communit vaults" should be "community vaults" in initialize function 3. Comment: "improvments" should be "improvements" above getHookTarget The error name typo is particularly important as it's part of the public interface.
Recommendations	Fix all typos: rename the error name in interface and implementation, and correct the comment typos.
Comments / Resolution	Acknowledged.

Issue_66	Missing address zero check in upgradePairImplementation
Severity	**Informational**
Description	The upgradePairImplementation function does not validate that implementation_ is not the zero address before updating the state variable. This is inconsistent with other functions like initialize, setCommunityVaultFactory, and setCustomVolatileDynamicFeeModule which all use _checkAddressZero.
Recommendations	Add a zero address check for these values.
Comments / Resolution	Acknowledged.

PairFees

The PairFees contract is a minimal, per-pool fee escrow contract that exists 1:1 with a Pair. Its purpose is to physically segregate swap fees from the Pair's reserves so the AMM invariant and reserve accounting remain "clean" while still letting LPs claim fees later. During swaps, the Pair contract transfers the fee amounts for token0/token1 into this escrow. When an LP claims fees via the Pair, the Pair calls claimFeesFor on this contract, which then transfers the requested token amounts to the recipient.

Privileged Functions

- claimFeesFor

Issue_67	Redundant factory parameter and unused import in PairFees
Severity	**Informational**
Description	The PairFees contract has two redundant elements: 1. _factory: This values is set as an internal value but never used anywhere or able to be read publicly. 2. The contract imports IPairFactory but is never used.
Recommendations	Consider removing the unnecessary/redundant code.
Comments / Resolution	Acknowledged.

Issue_68	Missing error message in require statement
Severity	**Informational**
Description	The claimFeesFor function uses a require statement without an error message: require(msg.sender == pair); When this check fails, it will revert with a generic error, making debugging more difficult.
Recommendations	Use a custom error in the require statement for better gas efficiency and clearer error handling.
Comments / Resolution	Acknowledged.

RouterV2

The RouterV2 contract is the periphery "router" that users and integrators call to interact with dexV2 pairs without needing to manually compute optimal deposit ratios, hop-by-hop swap amounts, or WETH wrapping.

The router delegates all pricing and invariant enforcement to the pair contracts: it computes amounts using each pair's getAmountOut, transfers inputs to the first pair, then calls pair.swap(...) hop-by-hop via _swap. It also includes "supporting fee-on-transfer tokens" variants for swap and remove-liquidity-ETH flows; these variants try to infer the real amount that arrived in a pair by comparing on-chain balances against reserves, then swap accordingly.

For ETH handling, it treats native ETH only as WETH: it wraps ETH into WETH on entry (deposit) and unwraps on exit (withdraw), and the receive function only accepts ETH from the WETH contract to prevent accidental sends.

Core Invariants:

INV 1: Every function with ensure(deadline) must revert if deadline < block.timestamp.
INV 2: The contract should only accept raw ETH via its receive when msg.sender == wETH.
INV 3: _addLiquidity must ensure amountADesired >= amountAMin and amountBDesired >= amountBMin, and the computed optimal amounts must respect the min constraints. Similarly, removeLiquidity must enforce amountA >= amountAMin and amountB >= amountBMin.
INV 4: After addLiquidityETH, any excess ETH above amountETH is refunded to the caller; after ETH-removal functions, WETH must be unwrapped and sent out so the router does not retain ETH except transiently during execution.

Issue_69	Stable pool mints via router can be blocked by donating tokens and syncing
Severity	**Medium**
Description	When minting stableswap liquidity via the router, the router calculates the optimal token addition ratio. *if (reserveA == 0 && reserveB == 0) {* *(amountA, amountB) = (amountADesired, amountBDesired);* *} else {* *uint amountBOptimal = quoteLiquidity(amountADesired, reserveA, reserveB);* This quoteLiquidity function however reverts if either reserve is 0. *require(reserveA > 0 && reserveB > 0, "BaseV1Router: INSUFFICIENT_LIQUIDITY");* So if one of the reserves in a pool is 0, liquidity cannot be minted to the stableswap pool via the router anymore. The issue is that when a new stableswap pool is created, both its reserves is 0, and anyone can donate a token to it and call sync to update it's reserves. This will put it in a state where one of it's reserves is 0 and the other one isnt. Say Alice creates a USDC/USDT pool. Bob immediately donates 1 USDC to the pool and then calls sync on it. Now the reserve of the pool is modified to be (1,0). Since one of the reserves is 0, liquidity cannot be added to it via the router anymore. Since this is the only router in the system, users will have to either use a different router similar to the uniswap V2 router, or risk losing tokens by directly sending tokens to the contract and then manually calling mint on it, which is not recommended.
Recommendations	Prevent calling the sync function in the pair contract if totalSupply==0.

Comments / Resolution	Fixed by following recommendation.

Issue_70	Stable pool mints via router can be blocked due to equal amount check
Severity	**Medium**
Description	When minting stableswap liquidity via the router, the router calculates the optimal token addition ratio. *if (reserveA == 0 && reserveB == 0) {* *(amountA, amountB) = (amountADesired, amountBDesired);* *} else {* *uint amountBOptimal = quoteLiquidity(amountADesired, reserveA, reserveB);* The issue is that in the stableswap pool, the initial liquidity has an extra restriction, where both tokens have to be supplied in the same amounts. *require(* *(_amount0 * 1e18) / decimals0 == (_amount1 * 1e18) / decimals1,* *"Pair: stable deposits must be equal"* *);* In an empty pool, if a user donates tokens in the wrong ratio and syncs, this will trigger this check and make liquidity additions via the router fail. Say Alice creates an USDC/USDT pool. Bob donates 1 USDC and 2 USDT to the pool and calls sync on it. The reserves are now (1,2). Alice now tries to add liquidity to it with 100 USDC, 100 USDT. The router will calculate the optimal liquidity addition to be (50 USDC, 100 USDT). However since these 2 numbers are not equal, the liquidity addition will trigger the _initialLiquidityCheck function

	(since totalSupply is still 0) and revert. So users will not be able to add liquidity to the pool via the router, and need to interact with the pool directly to fix it.
Recommendations	Prevent calling the sync function in the pair contract if totalSupply==0.
Comments / Resolution	Fixed by following recommendation.

Issue_71	_swapSupportingFeeOnTransferTokens logic in RouterV2 is flawed
Severity	**Medium**
Description	The _swapSupportingFeeOnTransferTokens function uses the getAmountOut function to calculate the output amount. This result is used to carry out the actual swap. The issue is that while the _swapSupportingFeeOnTransferTokens function takes in a specific route, the getAmountOut function can return values from either the volatile or the stable pair. *return amountStable > amountVolatile ? (amountStable, true) : (amountVolatile, false);* Thus even though the route can specify volatile pool, it can actually quote using the stable pool. This will lead to a revert during the actual swap, breaking functionality.
Recommendations	_swapSupportingFeeOnTransferTokens should quote the amount from the specified pair (stable/volatile) in the route instead of using whatever the better available rate is.
Comments / Resolution	Fixed by following recommendation.

Issue_72	Missing error messages in require statements
Severity	**Informational**
Description	Several require statements in RouterV2 lack error messages, making debugging difficult: - require(amountADesired >= amountAMin); in _addLiquidity - require(amountBDesired >= amountBMin); in _addLiquidity - require(IBaseV1Pair(pair).transferFrom(msg.sender, pair, liquidity)); in removeLiquidity When these checks fail, the transaction reverts with a generic error, making it difficult to identify the specific reason for failure during debugging.
Recommendations	Add descriptive error messages or use custom errors for better gas efficiency and clearer error handling.
Comments / Resolution	Acknowledged.

Issue_73	Use of assert instead of require
Severity	**Informational**
Description	The _addLiquidity function uses assert(amountAOptimal <= amountADesired); which is semantically incorrect. The assert function should only be used for invariants that should never be false, while require is for user input validation. This condition can legitimately fail and should use require instead.
Recommendations	Replace assert with require with an appropriate error message or custom error.
Comments / Resolution	Acknowledged.

Issue_74	Incorrect comment
Severity	**Informational**
Description	The following comment exists in the quoteAddLiquidity function. *// create the pair if it doesn't exist yet* This is actually not true, since this is a view only function and cannot create any pairs.
Recommendations	Remove the incorrect comment.
Comments / Resolution	Acknowledged.

UniswapV2PartialRouter

The UniswapV2PartialRouter contract is a thin compatibility wrapper around the RouterV2 contract. Its goal is to expose a subset of Uniswap V2–like router function signatures (e.g., swapExactTokensForTokens(amountIn, amountOutMin, path, to, deadline) and the addLiquidity/removeLiquidity family) while routing all logic through RouterV2. The key behavior difference is that it forces every "path hop" to be stable=false, meaning it only targets volatile pools.

No issues were identified in this contract.

VolatileDynamicFeeOnePool

The VolatileDynamicFeeOnePool contract is an upgradeable "dynamic fee module" intended to be plugged into the factory for a specific volatile pair, so the factory can ask it for the current swap fee in basis points. This contract implements a deterministic fee schedule: the fee starts at startFeePercentage (BPS), then after startTimestamp it decreases by decreaseStepBps every decreaseInterval seconds, and it never goes below finalFeePercentage. External systems (notably the factory) call getFee(pair_), which checks that pair_ matches the configured pair and then returns (true, fee) where fee is computed by / based on block.timestamp.

Privileged Functions

- setStartTimestamp

Core Invariants:

INV 1: For any timestamp, calculate(...) must always return a fee in the closed interval [finalFeePercentage, startFeePercentage].
INV 2: Holding all parameters fixed and assuming startTimestamp != 0, for t2 >= t1 >= startTimestamp, the returned fee must satisfy fee(t2) <= fee(t1).

Issue_75	No restriction on setStartTimestamp.
Severity	**Governance**
Description	The function setStartTimestamp can be called at any time by the admin to set the startTimestamp value. The issue with this approach is that once the feeschedule starts, the admin can still set startTimestamp to a different value. If the admin choses to increase the startTimestamp value, it will actually increase the fee rate instead of decreasing it, since the current timestamp is now closer to the startTimestamp and thus the fee should be closer to the starting value than the ending value. Thus the monotonic decreasing nature of the fee rate can actually be violated by the admins.
Recommendations	Consider restricting the time window when setStartTimestamp can be called, or documenting and acknowledging this behaviour.
Comments / Resolution	Acknowledged.

Issue_76	Unused InvalidCaller error definition
Severity	**Informational**
Description	The InvalidCaller error is defined in the contract but never used anywhere in the code. This is dead code that increases contract size unnecessarily.
Recommendations	Remove the unused error definition if not needed, or implement functionality that uses it if intended for future use.
Comments / Resolution	Acknowledged.

Issue_77	Unused IPair import
Severity	**Informational**
Description	The contract imports IPair but this import is never used anywhere in the contract code.
Recommendations	Remove the unused IPair import.
Comments / Resolution	Acknowledged.

Fees

FeesVaultFactoryUpgradeable

The FeesVaultFactoryUpgradeable contract serves as the central factory and configuration manager for fee vaults across all liquidity pools. It deploys new vault instances, stores the fee distribution configurations, and provides a hierarchical configuration system allowing different fee splits for different pools or creators.

Vault creation is handled by createVaultForPool, which deploys a new FeesVaultProxy pointing to the shared implementation and initializes it with the factory address and pool. Only addresses with WHITELISTED_CREATOR_ROLE can create vaults, and each pool can only have one vault. The creator address is recorded for each vault to support creator-specific configurations.

The factory implements a three-tier configuration hierarchy for fee distribution. When a vault requests its configuration via getDistributionConfig, the factory checks in order: first for a custom config specific to that vault, then for a config associated with the vault's creator, and finally falls back to the default config. This allows granular control where specific pools can have unique splits while most pools use sensible defaults.

Each DistributionConfig specifies a toGaugeRate (the percentage sent to the gauge for voter rewards) and parallel arrays of recipients and rates for additional protocol fee recipients. All rates are in basis points and must sum to exactly 10000 (100%). The _checkDistributionConfig function validates this constraint along with ensuring no zero addresses or rates.

Configuration management is restricted by role. FEES_VAULT_ADMINISTRATOR_ROLE can set the default config, custom configs for specific vaults, and creator-level configs. DEFAULT_ADMIN_ROLE can change the vault implementation, set the voter address, and reassign vault creators. Setting an empty config (all zeros) deletes a custom or creator config, causing the vault to fall back to the next tier.

Privileged Functions

- changeImplemenation
- setVoter
- setDefaultDistributionConfig
- setCustomDistributionConfig
- setDistributionConfigForCreator
- changeCreatorForFeesVaults
- createVaultForPool

Core Invariants:

INV 1: All distribution configurations must have rates summing to exactly 10000 basis points (100%). The _checkDistributionConfig function enforces this on every config update.
INV 2: Each pool can have at most one fee vault. Attempting to create a second vault for the same pool reverts with AlreadyCreated.
INV 3: Only addresses with WHITELISTED_CREATOR_ROLE can create new fee vaults.
INV 4: Configuration resolution follows strict priority: custom vault config, then creator config, then default config. A vault always receives exactly one applicable configuration.
INV 5: Recipient arrays and rate arrays within a config must have matching lengths, and no recipient address can be zero.
INV 6: No individual rate within a config can be zero. If a recipient should receive nothing, they should be removed from the config entirely.

Issue_78	Redundant import in FeesVaultFactoryUpgradeable
Severity	**Informational**
Description	The FeesVaultFactoryUpgradeable contract imports IPairIntegrationInfo but this interface is never used anywhere in the contract.
Recommendations	Remove the unused IPairIntegrationInfo import.
Comments / Resolution	Acknowledged.

FeesVaultProxy

The FeesVaultProxy contract implements a minimal upgradeable proxy for fee vault instances. Each fee vault is deployed as a proxy that delegates all calls to a shared implementation contract managed by the factory. This pattern allows all fee vaults to be upgraded simultaneously by changing the implementation address in the factory.

No issues were identified in this contract.

FeesVaultUpgradeable

The FeesVaultUpgradeable contract serves as a fee collection and distribution vault for liquidity pools. Each pool has its own vault instance that accumulates trading fees in the pool's token0 and token1. When claimed, fees are distributed according to a configurable split between the pool's gauge and other protocol recipients.

The vault is created through the FeesVaultFactory and stores references to its factory and associated pool. Trading fees from the pool accumulate in the vault as the two pool tokens. The distribution configuration is fetched from the factory at claim time, allowing the protocol to adjust fee splits without redeploying vaults.

The claimFees function distributes accumulated fees according to the factory's distribution config. The config specifies a toGaugeRate (percentage to the gauge in basis points out of 10000) and arrays of additional recipients with their respective rates. Access control depends on the gauge rate: if toGaugeRate is greater than zero, only the pool's registered gauge can call claimFees, and it must match the correct pool. If toGaugeRate is zero, only addresses with CLAIM_FEES_CALLER_ROLE can trigger distribution.

The internal _distributeFees function handles the actual transfers. It reads the vault's full balance for each token, calculates the gauge portion using the configured rate, transfers that amount to the caller (the gauge), then iterates through additional recipients and transfers their portions. Events are emitted for each distribution.

The emergencyRecoverERC20 function allows addresses with FEES_VAULT_ADMINISTRATOR_ROLE to recover any ERC20 tokens from the vault. This serves as a safety mechanism for recovering stuck or incorrectly sent tokens.

Privileged Functions

- claimFees
- emergencyRecoverERC20

Core Invariants:

INV 1: Only the gauge registered for this vault's pool can call claimFees when toGaugeRate is greater than zero. The pool-gauge relationship is verified through the Voter contract.
INV 2: When toGaugeRate is zero, only addresses with CLAIM_FEES_CALLER_ROLE can trigger fee distribution.
INV 3: The sum of toGaugeRate and all recipient rates should not exceed _PRECISION (10000) to avoid distributing more than 100% of fees. This is enforced by the factory configuration.
INV 4: Fee distribution uses the vault's current token balance at claim time, distributing all accumulated fees in a single transaction.
INV 5: The vault must only distribute fees for its associated pool. The pool address is immutable after initialization.

Issue_79	Emergency recover function can break fee distribution in FeesVaultUpgradeable
Severity	**Governance**
Description	The FeesVaultUpgradeable contract has an emergencyRecoverERC20 function that allows admins to withdraw any ERC20 tokens from the contract. While intended for emergency situations, this function can be misused to drain trading fees that should be distributed to LP providers and bribes. If fees are recovered before distribution, the fee distribution to bribes and other recipients will fail, affecting the entire reward ecosystem.
Recommendations	Add validation to prevent withdrawal of tokens with pending distributions or acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_80	Incorrect NatSpec documentation in FeesVaultUpgradeable
Severity	**Informational**
Description	The FeesVaultUpgradeable contract has incorrect NatSpec documentation. The @title states "Fees Vault Factory" and the @dev comment describes it as a "Factory contract for creating and managing fees vault instances." However, this contract is actually the fees vault implementation, not the factory. The factory contract is FeesVaultFactoryUpgradeable.
Recommendations	Update the NatSpec documentation to correctly describe the contract as a fees vault implementation that manages fee distribution for a specific pool, rather than a factory contract.
Comments / Resolution	Acknowledged.

Issue_81	Discrepancy in fee distribution for killed gauges
Severity	**Informational**
Description	There is a logic discrepancy between how the VoterUpgradeableV2 and FeesVaultUpgradeable contracts handle killed gauges. 1. The Voter blocks collection: In Voter.distributeFees, the contract explicitly checks if a gauge isAlive. If a gauge has been killed, the Voter skips it, preventing automated fee collection. 2. The FeesVault allows collection: In FeesVault.claimFees, the contract only verifies that the caller is a registered gauge and matches the pool. It does not check the alive status. As a result, fees can still be pulled if someone calls claimFees directly on the killed Gauge contract, bypassing the Voter's intended restriction.
Recommendations	Unify the fee collection policy across both contracts. If the intention is to stop all activity for killed gauges, FeesVault.claimFees should be updated to verify that the gauge isAlive in the Voter before distributing funds.
Comments / Resolution	Acknowledged.

Gauges

GaugeFactoryUpgradeable

The GaugeFactoryUpgradeable contract serves as the factory for deploying gauge instances that distribute protocol emissions to liquidity providers. Each gauge is associated with a specific liquidity pool and manages the distribution of NEST rewards to stakers based on their LP token deposits and veNFT voting weight.

The factory deploys gauges as GaugeProxy instances that delegate to a shared implementation contract. This allows all gauges to be upgraded simultaneously by changing the gaugeImplementation address. The factory stores references to the voter contract and an optional merklGaugeMiddleman for integration with Merkl distribution systems.

Privileged Functions

- createGauge
- changeImplementation
- setMerklGaugeMiddleman

Core Invariants:

INV 1: Only the voter contract or the factory owner can create new gauges. This ensures gauges are created exclusively through the official gauge creation flow in the Voter contract.
INV 2: The gaugeImplementation address must be non-zero at initialization. All deployed GaugeProxy instances delegate to this address.
INV 3: The voter address must be non-zero at initialization and cannot be changed after deployment.
INV 4: Each createGauge call deploys a new independent proxy. There is no restriction on creating multiple gauges, though the Voter contract enforces one gauge per pool.
INV 5: Implementation changes via changeImplementation affect all existing gauges immediately since they fetch the implementation from the factory on each call.

Issue_82	Missing event emission merklGaugeMiddleman is set in setMerklGaugeMiddleman
Severity	**Informational**
Description	In the setMerklGaugeMiddleman function, the merklGaugeMiddleman value is set, however no event is emitted indicating the same.
Recommendations	Emit an event in the setMerklGaugeMiddleman function.
Comments / Resolution	Acknowledged.

GaugeProxy

The GaugeProxy contract implements a minimal upgradeable proxy for gauge instances. Each gauge is deployed as a proxy that delegates all calls to a shared implementation contract managed by the GaugeFactory. This pattern allows all gauges across the protocol to be upgraded atomically by changing the implementation address in the factory.

No issues were identified in this contract.

GaugeUpgradeable

The GaugeUpgradeable contract is the core implementation for liquidity mining gauges. Users stake LP tokens to earn NEST emissions proportional to their share of the gauge's total deposits. The gauge receives emissions from the Voter contract and distributes them to stakers over weekly periods using a time-weighted reward accumulator pattern.

Staking is handled through deposit and withdrawal functions. When users deposit LP tokens, their balance is recorded, and the gauge's total supply increases. The updateReward modifier ensures pending rewards are calculated and credited before any balance changes. The gauge can optionally integrate with a gaugeRewarder contract that receives callbacks on deposits, withdrawals, and claims for external reward distribution.

Reward calculation follows the standard rewardPerToken model. The rewardRate determines how many tokens are distributed per second across all stakers. Each user's earned rewards are calculated as their balance multiplied by the difference between the current rewardPerTokenStored and their last recorded userRewardPerTokenPaid. Users claim rewards via getReward, which transfers accumulated NEST to them.

The Voter contract calls notifyRewardAmount to inject new emissions into the gauge. If Merkl distribution is enabled, rewards are forwarded to the merklGaugeMiddleman for off-chain distribution. Otherwise, the new rewards are added to any remaining rewards from the current period, and a new 7-day DURATION begins with an updated rewardRate.

The claimFees function collects trading fees from the fee vault and, for V2 gauges, directly from the pair contract. Collected fees in token0 and token1 are forwarded to the internal_bribe contract, where they become available as bribes to veNFT voters who voted for this gauge's pool.

The contract supports an emergency mode that allows users to withdraw their LP tokens without any reward calculations or external calls, providing a safety escape hatch if the normal withdrawal path becomes blocked.

Privileged Functions

- setDistribution
- setMerklGaugeMiddleman
- setIsDistributEmissionToMerkle
- setGaugeRewarder
- setFeeVault
- setInternalBribe
- activeEmergencyMode
- stopEmergencyMode
- notifyRewardAmount
- getReward

Core Invariants:

INV 1: Only the DISTRIBUTION address (the Voter contract) can call notifyRewardAmount to inject new emissions into the gauge.
INV 2: The sum of all user balances must equal _totalSupply at all times.
INV 3: The rewardRate must never exceed the gauge's token balance divided by DURATION to prevent reward insolvency.
INV 4: Normal deposits and withdrawals are blocked during emergency mode. Only emergencyWithdraw and emergencyWithdrawAmount are available.
INV 5: When isDistributeEmissionToMerkle is true, all emissions are forwarded to the merklGaugeMiddleman rather than being distributed directly through the gauge.
INV 6: Fees claimed via claimFees are always forwarded to the internal_bribe contract. The gauge does not retain trading fees.
INV 7: The DURATION is fixed at 7 days (604800 seconds). Each notifyRewardAmount call resets the period finish to the current timestamp plus DURATION.

Issue_83	Rewards routed to MerklGaugeMiddleman are permanently lost if below threshold
Severity	**Low**

Description	Reward tokens are permanently lost when emissions routed to the MerklGaugeMiddleman fail to meet the minimum distribution threshold, as the returned tokens are never accounted for by the Gauge.
	In GaugeUpgradeable, when a gauge is configured to use the Merkl integration (isDistributeEmissionToMerkle == true), the contract transfers its entire reward allocation to the MerklGaugeMiddleman. The Middleman then checks if the amount meets the Merkl-specific rewardTokenMinAmounts. If the amount is too low, the Middleman transfers the tokens back to the Gauge:
	
	*if (amount_ > creatorCache.rewardTokenMinAmounts(address(tokenChache)) * params.numEpoch) {*
	* uint256 distributionAmount = creatorCache.createDistribution(params);*
	* emit CreateDistribution(msg.sender, gauge_, amount_, distributionAmount);*
	} else {
	* tokenChache.safeTransfer(msg.sender, amount_); // Tokens sent back to Gauge*
	}
	
	However, the GaugeUpgradeable logic does not have a mechanism to handle these returned tokens.
	
	This issue is marked as low severity since this contract will not be used by the protocol team.

Recommendations	Consider implementing a mechanism that accounts for the returned tokens.
Comments / Resolution	Acknowledged.

Issue_84	Inconsistent _periodFinish updates enable reward theft
Severity	**Medium**
Description	The notifyRewardAmount function incorrectly extends the internal _periodFinish even when rewards are routed to an external distributor (Merkl). This allows the Gauge to accrue "ghost" rewards using a stale rewardRate, leading to contract insolvency and potential theft of user funds when isDistributeEmissionToMerkle is set from false to true. If a Gauge previously had a rewardRate of 10 tokens/sec and governance switches to Merkl, a subsequent call to notifyRewardAmount will extend the _periodFinish by another week. Even though the new tokens were sent to Merkl, the Gauge will continue to calculate earned() rewards for users at 10 tokens/sec for the new week. Since the Gauge no longer possesses these tokens (they are in Merkl), the users will eventually drain the Gauge's balance, stealing rewards from previous depositors who have not yet claimed.
Recommendations	Consider not increasing _periodFinish when isDistributeEmissionToMerkle is true.
Comments / Resolution	Acknowledged.

Issue_85	Lack of reward recovery for unallocated or precision loss tokens
Severity	**Low**
Description	The GaugeUpgradeable contract lacks a mechanism to recover or redistribute reward tokens that are not allocated to users due to periods of zero total supply or precision loss during rate calculations. There are two primary scenarios where rewards become permanently trapped: 1. Zero Total Supply: If notifyRewardAmount is called while _totalSupply is 0, the rewards meant for that period are never accrued by the rewardPerToken logic. The lastUpdateTime advances, but the "reward-per-token" index remains stagnant. 2. Precision Loss: The calculation *rewardRate = (reward + leftover) / DURATION* performs integer division. If the numerator is smaller than the DURATION (604,800 seconds), the rewardRate rounds down to zero. Unlike the BribeUpgradeable contract, which features functionality to recover ERC20 tokens, the GaugeUpgradeable contract provides no way for governance to retrieve these unallocated tokens.
Recommendations	Consider implementing a restricted recoverERC20 function in the Gauge contract to allow governance to claw back unallocated rewards.
Comments / Resolution	Acknowledged.

Issue_86	Rewards would be lost in emergency mode in gauge
Severity	**Low**
Description	When the Gauge contract is in emergency mode updateRewards is not invoked during emergencyWithdraw , this means that the rewards corresponding to that user invoking emergencyWithdraw are lost and stuck forever in the Gauge contract.
Recommendations	Consider adding a recover function where the reward amount can be transferred out from the Gauge.
Comments / Resolution	Acknowledged.

Issue_87	State desync via setInternalBribe bricks fee distribution
Severity	**Low**
Description	In GaugeUpgradeable, the setInternalBribe function allows the owner to change the destination for LP fees. When fees are claimed, they are sent to this internal_bribe address. However, the VoterUpgradeableV2 contract caches the original internal bribe address in its gaugesState mapping during gauge creation. The Voter uses this cached address to manage user voting power deposits and withdrawals whenever a user calls vote, poke, or reset. Because the Gauge sends rewards to the new address while the Voter continues to manage user balances in the old address, the two states diverge. As a result, Users who vote for the gauge to earn LP fees will receive nothing, as their voting power is registered in the old bribe contract which no longer receives fee inflows
Recommendations	Consider implementing a function in the Voter to update the internal bribe in the gaugesState mapping.
Comments / Resolution	Acknowledged.

Issue_88	Reward rate validation does not account for unclaimed rewards in balance
Severity	**Informational**
Description	The reward rate validation in the notifyRewardAmount function checks *require(rewardRate <= balance / (DURATION), "Provided reward too high")*, where balance is the total balance of the reward token in the contract. This check fails to account for the fact that the balance includes both rewards that are yet to be distributed (leftover from previous periods) and rewards that have not been claimed by users. This check is not needed at all as the contract should rely on the actual token balance when distributing rewards, and any insufficient balance will naturally cause transfers to fail.
Recommendations	Remove the validation check entirely, as it is unnecessary and incorrect.
Comments / Resolution	Acknowledged.

Issue_89	Redundant import
Severity	**Informational**
Description	The GaugeUpgradeable contract imports IPairInfo interface but does not use it anywhere in the contract.
Recommendations	Remove the redundant import.
Comments / Resolution	Acknowledged.

Issue_90	gaugeRewarder and IRewarder interface are unused
Severity	Informational
Description	The GaugeUpgradeable contract imports IRewarder interface and defines a gaugeRewarder address variable that is used in multiple functions to call IRewarder(gaugeRewarder).onReward. However, there is no contract implementation in the codebase that implements the IRewarder interface or the onReward function. The actual gaugeRewarder contract is different and does not implement an onReward function.
Recommendations	Implement the gaugeRewarder contract or remove the function call since no such contract is present.
Comments / Resolution	Acknowledged.

Issue_91	Missing error message in require statement for merklGaugeMiddleman check
Severity	Informational
Description	In the setIsDistributeEmissionToMerkle function, the require statement checking that merklGaugeMiddleman != address(0) is missing an error message. However, in the initialize function, the same check uses the error message "not setup merklGaugeMiddleman". This inconsistency means that when the require statement reverts in setIsDistributeEmissionToMerkle, it will revert with a generic error instead of providing a clear, descriptive message like in initialize.
Recommendations	Add the same error message to the setIsDistributeEmissionToMerkle function.
Comments / Resolution	Acknowledged.

Issue_92	Missing event emissions
Severity	**Informational**
Description	The setDistribution, setMerklGaugeMiddleman, setIsDistributeEmissionToMerkle, setGaugeRewarder, setFeeVault, and setInternalBribe functions all make state changes but does not emit any event.
Recommendations	Emit events in the above functions to indicate the state change.
Comments / Resolution	Acknowledged.

PerpetualsGaugeUpgradeable

The PerpetualsGaugeUpgradeable contract is a specialized gauge designed for perpetual trading incentives. Unlike standard LP gauges that handle staking and time-weighted reward distribution internally, this gauge acts as a passthrough that forwards all emissions to an external rewarder contract responsible for distributing rewards to perpetual traders.

The gauge receives emissions from the Voter contract through notifyRewardAmount, which is restricted to the DISTRIBUTION address (the Voter). When emissions arrive, the gauge transfers the reward tokens from the Voter to itself, approves the rewarder contract, and calls the rewarder's notifyRewardAmount to forward the full amount. The rewarder contract then handles the actual distribution logic based on trading activity or other criteria specific to perpetual markets.

Privileged Functions

- notifyRewardAmount

Core Invariants:

INV 1: Only the DISTRIBUTION address (the Voter contract) can call notifyRewardAmount to send emissions to this gauge.
INV 2: All reward tokens received via notifyRewardAmount are immediately forwarded to the rewarder contract. The gauge does not retain emissions.
INV 3: The reward token address is fixed at initialization and cannot be changed. Calls with a different token address revert with IncorrectRewardToken.
INV 4: The rewarder, rewardToken, and DISTRIBUTION addresses must all be non-zero at initialization.
INV 5: The claimFees function always returns (0, 0) as perpetual gauges do not collect LP trading fees.
INV 6: The getReward function performs no operations. Reward distribution to users is handled entirely by the external rewarder contract.

Issue_93	Typo in interface name: Reciever should be Receiver
Severity	**Informational**
Description	The PerpetualsGaugeUpgradeable contract contains a typo in the interface name where "Reciever" is used instead of "Receiver". This typo appears in the import statement and when casting the rewarder address to the interface type.
Recommendations	Correct the spelling from "Reciever" to "Receiver".
Comments / Resolution	Acknowledged.

PerpetualsTradersRewarderUpgradeable

The PerpetualsTradersRewarderUpgradeable contract handles the distribution of NEST rewards to perpetual traders using an off-chain signature-based claim system. It receives emissions from the PerpetualsGauge and allows traders to claim their earned rewards by presenting a valid signature from an authorized backend signer.

The gauge calls notifyRewardAmount to deposit reward tokens into this contract. The function verifies the caller is the registered gauge and that the token matches the expected reward token, then transfers the tokens in and increments totalReward. This provides a running total of all emissions directed to perpetual trading incentives.

Privileged Functions

- setSigner
- notifyRewardAmount

Core Invariants:

INV 1: Only the registered gauge address can call notifyRewardAmount to deposit reward tokens.
INV 2: Users can only claim rewards with a valid signature from the current signer address. Invalid or tampered signatures revert with InvalidSignature.
INV 3: The signed amount represents cumulative entitlement, not per-claim amounts. A user's actual payout equals the signed amount minus their previously claimed amount.
INV 4: A user cannot claim the same amount twice. If the signed amount is less than or equal to the already claimed amount, the transaction reverts with AlreadyClaimed.
INV 5: When the signer is set to address(0), all claims are disabled and revert with ClaimDisabled.
INV 6: The totalClaimed value must never exceed totalReward, assuming the backend correctly limits signed amounts to available rewards.
INV 7: The sum of all values in the claimed mapping should equal totalClaimed.

Issue_94	Changing signer in PerpetualsTradersRewarder can allow unauthorized fund drainage
Severity	**Governance**
Description	The PerpetualsTradersRewarderUpgradeable contract relies on an off-chain signer to authorize reward claims via EIP-712 signatures. The owner can change the signer address at any time using setSigner. If a malicious or compromised owner sets the signer to an address they control, the new signer can generate valid signatures for any claim amount to arbitrary addresses, effectively draining all reward tokens from the contract.
Recommendations	Add a timelock for signer changes to allow users to claim pending rewards before the change takes effect.
Comments / Resolution	Acknowledged.

Integration

AlgebraNESTPriceProviderUpgradeable

The AlgebraNESTPriceProviderUpgradeable contract provides on-chain price data for the NEST token by querying an Algebra V3 pool. It returns how much NEST can be obtained for one USD based on the pool's current tick, allowing other protocol components to perform value conversions between USD-denominated stablecoins and NEST.

The contract is explicitly marked as UNSAFE in its title and documentation because it uses the pool's instantaneous spot price rather than a time-weighted average price (TWAP). This makes the returned price susceptible to manipulation through large swaps or flash loans within the referenced Algebra pool. The documentation warns that it should only be used in contexts where the cost of price manipulation exceeds any potential profit from doing so.

Core Invariants:

INV 1: The pool, NEST, and USD addresses are fixed at initialization and cannot be changed afterward.
INV 2: Price queries revert with PoolIsLocked if the Algebra pool is currently locked during a swap transaction.
INV 3: ONE_USD is calculated from the USD token's decimals at initialization and represents one full unit of the stablecoin.
INV 4: The returned price reflects the instantaneous spot price at the current tick, not a time-weighted average. This is inherently manipulable.
INV 5: The uint256 to uint128 conversion for ONE_USD reverts with UnsafeCast if overflow would occur, though this is unlikely for standard stablecoin decimals.
INV 6: The contract does not validate that the pool actually contains the NEST and USD tokens. Misconfiguration at initialization could produce incorrect prices.

Issue_95	Algebra-based on-chain price can be manipulated and must not be used for value-extracting logic
Severity	**Low**
Description	AlgebraNESTPriceProviderUpgradeable derives the NEST price from the Algebra pool via: *return OracleLibrary.getQuoteAtTick(currentTick(), _toUint128(ONE_USD), USD, NEST);* This price is based on the current tick and can be manipulated via concentrated liquidity and short-term trading around the call. The contract's NatSpec correctly acknowledges this limitation, and in the current codebase this price is only used inside the Boost contract, where price manipulation does not yield a profitable payoff for users without incurring a larger, asymmetric cost. However, if this price feed were to be reused elsewhere (e.g. for swaps, liquidations, collateral checks, or any function that directly extracts or transfers value based on the returned price), it would become a critical manipulation vector. This issue is marked as low severity since this contract will not be used by the protocol team.
Recommendations	Keep using this price feed only in contexts like the current Boost logic where manipulation is not economically viable. The Algebra price is **not** a secure oracle and must not be used for value-extracting decisions (swaps, liquidations, collateral, slippage checks).
Comments / Resolution	Acknowledged.

ManualNESTPriceProvider

The ManualNESTPriceProvider contract implements a simple administrator-controlled price feed for the NEST token. Unlike the Algebra-based price provider that reads from a DEX pool, this contract relies on a trusted owner to manually set and update the price of NEST in USD terms.

The contract stores a single price value representing how much NEST corresponds to one USD. The owner sets this value through setNestPrice, and external contracts query it via getUsdToNESTPrice. This design is useful during early protocol stages before sufficient on-chain liquidity exists, for testing environments, or as a fallback when automated price feeds are unavailable.

Privileged Functions

- setNestPrice

Core Invariants:

INV 1: Only the contract owner can update the price via setNestPrice.
INV 2: If the price is zero, getUsdToNESTPrice reverts with PriceNotSetup. A zero price is treated as uninitialized.
INV 3: The price represents how much NEST corresponds to one USD, matching the interface contract expected by consumers.
INV 4: Price updates are recorded via SetPrice events, providing a complete history of all changes on-chain.
INV 5: The contract has no mechanism to prevent the owner from setting arbitrary prices. Consumers must trust the owner to maintain accurate pricing.
INV 6: This contract is not upgradeable. The price feed address must be updated in dependent contracts if migration to a different provider is needed.

Issue_96	Manual price provider allows arbitrary price manipulation
Severity	**Governance**
Description	The ManualNESTPriceProvider contract allows the owner to set the NEST token price at any value via setNestPrice. This price is used by VeBoostUpgradeable to calculate boost rewards and potentially other protocol components. A malicious owner could set an extremely high or low NEST price to manipulate boost rewards, extract value from the boost mechanism, or front-run user transactions by changing the price.
Recommendations	Consider adding a timelock for price changes.
Comments / Resolution	Acknowledged.

MerklGaugeMiddleman

The MerklGaugeMiddleman contract serves as a bridge between the protocol's gauge system and Angle Protocol's Merkl off-chain reward distribution infrastructure. When gauges are configured to use Merkl distribution instead of direct staking rewards, they forward their emissions to this middleman, which then creates distributions on the Merkl DistributionCreator contract.

The contract is based on Angle Protocol's MerklGaugeMiddleman implementation and adapts it for the NEST protocol. At construction, it stores references to the reward token (NEST) and the Merkl DistributionCreator, accepts the Merkl terms and conditions, and grants maximum token allowance to the DistributionCreator for seamless distribution creation.

Privileged Functions

- setGauge

Core Invariants:

INV 1: Only the owner can configure gauge distribution parameters via setGauge.
INV 2: A gauge must have valid parameters set (non-zero uniV3Pool) before notifyReward can create a distribution for it.
INV 3: If the reward amount is below Merkl's minimum threshold for the configured number of epochs, the tokens are refunded to the caller rather than creating a distribution.
INV 4: The reward token configured for each gauge must match the contract's stored token address and must be whitelisted by the DistributionCreator.
INV 5: The contract maintains maximum allowance to the DistributionCreator. The setNestAllowance function exists to restore allowance if it somehow decreases.
INV 6: Distribution parameters are overwritten completely when setGauge is called. There is no partial update mechanism.

Issue_97	Anyone can drain tokens via refund path when reward amount is below minimum threshold
Severity	**Low**
Description	In MerklGaugeMiddleman._notifyReward, if amount_ is non-zero but does not meet the minimum required threshold, the balance is refunded to the caller. *if (amount_ > creatorCache.rewardTokenMinAmounts(address(tokenChache)) * params.numEpoch) {* * uint256 distributionAmount = creatorCache.createDistribution(params);* * emit CreateDistribution(msg.sender, gauge_, amount_, distributionAmount);* *} else {* * tokenChache.safeTransfer(msg.sender, amount_);* *}* This logic refunds the entire amount_ back to msg.sender when the amount is too small to create a distribution. However: - When amount_ == 0, the function first sets amount_ = tokenChache.balanceOf(address(this)), effectively using the **entire contract balance**. - If this full-balance amount_ is still below rewardTokenMinAmounts * params.numEpoch, the else branch executes and **transfers all tokens held by the contract to the caller**. This means that any caller can invoke _notifyReward (via the public entrypoint that uses it) with amount_ = 0 in a configuration where the merkl minimum threshold is higher than the contract's current balance, and unconditionally drain all tokens from the middleman contract. There is no access control or whitelist on who can trigger this refund path. This issue is marked as low severity since this contract will not be used by the protocol team.
Recommendations	Remove the refund branch entirely or restrict it to prevent such drains.

Comments / Resolution	Acknowledged.

OpenOceanVeNftDirectBuyer

The OpenOceanVeNftDirectBuyer contract enables users to purchase veNFT positions in a single transaction by swapping any token through the OpenOcean DEX aggregator and immediately locking the resulting NEST into VotingEscrow. This simplifies the user experience by combining what would otherwise be separate swap and lock operations.

The directVeNftPurchase function orchestrates the entire flow. It accepts OpenOcean swap parameters (caller, description, and call data) plus VotingEscrow lock parameters (duration, recipient, boost flag, permanent lock flag, and optional managed NFT attachment). The function validates that the destination token of the swap matches the expected NEST token and that the destination receiver is either this contract or address zero.

For ERC20 source tokens, the contract pulls the swap amount from the caller, approves the OpenOcean exchange, and executes the swap. For ETH swaps, the contract forwards the msg.value to OpenOcean. After the swap completes, the contract verifies that the actual received amount matches the reported output to prevent discrepancies.

Any unused source tokens remaining after the swap (from partial fills or slippage protection) are returned to the caller. The contract then approves VotingEscrow for the full received NEST amount and calls createLockFor to mint the veNFT to the specified recipient with the requested parameters.

Privileged Functions

- rescueFunds

Core Invariants:

INV 1: The destination token of the swap must match the contract's configured token (NEST). Swaps to other tokens revert with InvalidDstToken.
INV 2: The destination receiver in the swap description must be either this contract or address zero. Other receivers revert with InvalidDstReceiver.
INV 3: The actual token balance increase after the swap must exactly match the amount reported by the swap function. Mismatches revert with InvalidOutputAmount.
INV 4: Permit functionality on source tokens is not supported and reverts with PermitNotSupported to avoid potential security issues.
INV 5: All NEST tokens received from the swap are used to create the veNFT. The contract does not retain any NEST from successful purchases.
INV 6: Any excess source tokens not consumed by the swap are returned to the caller.
INV 7: ETH is identified by address 0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeEEeE or address zero, following OpenOcean conventions.

Issue_98	Unused IERC20Permit import in OpenOceanVeNftDirectBuyer
Severity	**Informational**
Description	The OpenOceanVeNftDirectBuyer contract imports IERC20Permit from OpenZeppelin but does not use it anywhere in the contract. The contract does not support permit-based approvals and explicitly reverts when desc_.permit is non-empty.
Recommendations	Remove the unused IERC20Permit import to reduce contract size and improve code clarity.
Comments / Resolution	Acknowledged.

Nest

BaseManagedNFTStrategyUpgradeable

The BaseManagedNFTStrategyUpgradeable contract is an abstract upgradeable base contract for strategies that manage a single veNFT position with voting and reward capabilities. It holds a managedTokenId representing an NFT in VotingEscrow that the strategy controls. The contract can cast governance votes on behalf of the managed NFT and claim rewards and bribes accumulated by that position. Access control is delegated to an external ManagedNFTManager, which defines admins and authorized users.

Privileged Functions

- attachManagedNFT
- setName
- setCreator
- setDescription
- vote

Core Invariants:

INV 1: A managedTokenId can only be attached once and must be a valid managed NFT owned by this contract.
INV 2: Only addresses recognized as admins by the ManagedNFTManager can attach NFTs or modify strategy metadata. INV 3: Only addresses authorized for the specific managedTokenId can cast votes through the strategy.

No issues were identified in this contract.

CompoundVeNESTManagedNFTStrategyFactoryUpgradeable

The CompoundVeNESTManagedNFTStrategyFactoryUpgradeable contract is a factory for deploying CompoundVeNESTManagedNFTStrategy instances alongside their paired VirtualRewarder contracts. It stores implementation addresses, a managedNFTManager reference, and a routerV2PathProvider that gets passed to newly created instances.

When createStrategy is called, the factory deploys both a StrategyProxy and a VirtualRewarderProxy, initializes them with the stored configuration, and links them together. The proxies delegate to the implementation addresses stored in this factory, allowing admins to change implementations for future deployments without affecting existing ones.

Privileged Functions

- createStrategy
- changeVirtualRewarderImplementation
- changeStrategyImplementation
- setRouterV2PathProvider

Core Invariants:

INV 1: Only accounts with STRATEGY_CREATOR_ROLE can deploy new strategy and rewarder pairs.
INV 2: Only accounts with DEFAULT_ADMIN_ROLE can change implementation addresses or the router path provider.
INV 3: All stored addresses must be non-zero; the factory rejects zero addresses on initialization and updates.

No issues were identified in this contract.

CompoundVeNESTManagedNFTStrategyUpgradeable

The CompoundVeNESTManagedNFTStrategyUpgradeable contract is a concrete strategy implementation that manages a veNFT position and auto-compounds rewards back into it. It extends BaseManagedNFTStrategyUpgradeable for core NFT management and SingelTokenBuybackUpgradeable for token swap capabilities.

Users attach their veNFTs to this strategy via the ManagedNFTManager, which calls onAttach to register them in the virtualRewarder. When detaching via onDettach, the strategy enforces a time-lock window at the beginning of each weekly epoch to prevent vote manipulation. The lock duration can be set per-strategy or fall back to a manager default.

The strategy can compound in two ways: compound deposits the contract's NEST balance into the managed veNFT, while compoundVeNFTs and compoundVeNFTsAll merge other veNFTs held by the strategy into the managed one. Both paths notify the virtualRewarder of the increased locked amount, so rewards can be distributed proportionally to attached users.

Recovery functions allow admins or authorized users to retrieve ERC20 tokens and veNFTs sent to the contract, with restrictions preventing recovery of the NEST token, buyback route tokens, and the managedTokenId unless specific strategy flags override these protections.

Privileged Functions

- onAttach
- onDettach
- setDetachmentLockDuration
- setRouterV2PathProvider
- erc20Recover
- erc721Recover
- claimBribesWithERC20Recover
- claimBribesWithTokensRecover

Core Invariants:

INV 1: The managedTokenId can never be recovered or merged into another veNFT.

INV 2: Detachment is blocked while within the lock window at the start of each weekly epoch.
INV 3: Compounding veNFTs must result in a non-zero increase to the managed position's locked amount.
INV 4: NEST token and tokens in buyback routes cannot be recovered unless the IGNORE_RESTRICTIONS_ON_RECOVER_TOKENS flag is set.
INV 5: veNFT recovery from votingEscrow is blocked unless the IGNORE_RESTRICTIONS_ON_RECOVER_VE_NFT_TOKENS flag is set.

Issue_99	compoundVeNFTs fails with default managed veNFT state
Severity	**Low**
Description	When a managed veNFT is created, it is created with 0 underlying balance. This token actually fails the _checkExistAndNotExpired check in the VE contract, since it is not attached and has no balance. *if (state.locked.amount == 0) {* *revert TokenNotExist();* *}* Due to this reason, tokens cannot even be merged into it. Thus the _compoundVeNFTs function is unable to merge any veNFT tokens into the managed veNFT contract, since the target contract fails the existence check according to the protocol.
Recommendations	_checkExistAndNotExpired should pass in case of a managed NFT.
Comments / Resolution	Acknowledged.

Issue_100	compoundVeNFTs cannot merge veNFTs which are attached to the strategy managedNFT
Severity	**Medium**
Description	The compoundVeNFTs function is used to merge veNFTs present in the contract to the managed veNFT controlled by the contract. However in certain situations this merge can fail. During the compounding process, there is a check to make sure the managed veNFT gained value. *uint256 balanceAfter = uint256(int256(votingEscrowCache.getNftState(managedTokenIdCache).locked.amount));* *uint256 compoundRewards = balanceAfter - balanceBefore;* *if (compoundRewards == 0) {* *revert ZeroCompoundVeNFTsReward();* *}* This check can block legitimate merges/compounding. Lets assume the strategy contract controls the managed NFT with tokenId 100. If a veNFT is present in the contract which is already attached to this tokenId 100, it cannot be merged. This is because attached veNFTs have a balance of 0, and the balance is actually carried by the attachment target. So in this case, the veNFT will have a balance of 0, and its actual balance will already be present in the managedTokenId. Thus if the token is attempted to be merged, the compoundRewards will evaluate to 0, since the managedTokenId did not gain any value, since it just went from controlling the veNFTs balance to owning it via merge. Thus the veNFT merge will fail.
Recommendations	Consider skipping the compoundRewards check in this situation, or document and acknowledge this behaviour.
Comments / Resolution	Fixed by following recommendation.

Issue_101	compoundVeNFTsAll can be DOSed
Severity	Low
Description	The compoundVeNFTsAll is responsible for merging all available veNFT tokens in the contract into the managed token. However this function can be broken in multiple ways. If a large number of veNFTs are sent to this contract, then the loop going over all of them will consume more gas than the block gas limit and thus will never be able to execute. If expired veNFTs are present in the contract, they can never be merged since they will fail the _checkExistAndNotExpired check. If veNFTs which have voted are present in the contract, they can only be merged within the voting windows. Otherwise, the _resetVotesIfVoted function will revert.
Recommendations	Consider documenting and acknowledging the failure conditions of the compoundVeNFTsAll function.
Comments / Resolution	Acknowledged.

Issue_102	Managed NFT voting power is stale after compounding
Severity	**Low**
Description	In CompoundVeNESTManagedNFTStrategyUpgradeable, the functions compound and _compoundVeNFTs are used to increase the locked amount of the strategy's managedTokenId. However, neither the Strategy nor the underlying VotingEscrow contract triggers an update in the Voter contract to reflect the new balance. In the Voter contract, voting power for an epoch is "snapshotted" when _vote or _poke is called. If the balance increases via a direct deposit or a merge, the weightsPerEpoch and Bribe _balances associated with the Managed NFT remain at their pre-compounding levels.
Recommendations	Consider calling voter.poke(managedTokenId) at the end of the compound and _compoundVeNFTs functions. However this can only be done within the voting window. Alternatively, consider documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_103	Redundant duplicate import of LibStrategyFlags
Severity	**Informational**
Description	In CompoundVeNESTManagedNFTStrategyUpgradeable, the LibStrategyFlags library is imported twice, which is redundant.
Recommendations	Remove the duplicate LibStrategyFlags import.
Comments / Resolution	Acknowledged.

ManagedNFTManagerUpgradeable

The ManagedNFTManagerUpgradeable contract serves as the central registry and coordinator for managed NFTs and user NFT attachments within the protocol. It tracks which user veNFTs are attached to which managed veNFTs and coordinates the attach/detach lifecycle between the Voter, VotingEscrow, and Strategy contracts.

When a user attaches their veNFT to a managed position, the Voter calls onAttachToManagedNFT, which validates that the managed token is active, records the attachment in tokensInfo, coordinates with VotingEscrow to transfer voting power, and notifies the strategy. Detachment via onDettachFromManagedNFT reverses this process and includes any accrued rewards.

The contract maintains separate mappings for user token state (TokenInfo tracking attachment status and amounts) and managed token state (ManagedTokenInfo tracking if a token is managed, disabled, and its authorized user). Admins can create new managed NFTs, set authorized users per managed token, toggle disabled states, whitelist NFTs, and configure strategy flags. A configurable detachment lock duration allows strategies to enforce a time window at epoch start where detachment is blocked.

Privileged Functions

- createManagedNFT
- setStrategyFlags
- setAuthorizedUser
- toggleDisableManagedNFT
- setWhitelistedNFT
- setDefaultDetachmentLockDuration
- onAttachToManagedNFT
- onDettachFromManagedNFT
- onDepositToAttachedNFT

Core Invariants:

INV 1: Only the Voter contract can trigger user attach and detach operations.
INV 2: Only the VotingEscrow contract can notify of deposits to attached NFTs.
INV 3: Users cannot attach to or deposit into a disabled managed NFT.
INV 4: A managed NFT cannot itself be attached to another managed NFT.
INV 5: A user NFT can only be attached to one managed NFT at a time and cannot re-attach while already attached.
INV 6: The default detachment lock duration cannot exceed 6 days.

Issue_104	Unused AlreadyAttached error in ManagedNFTManager
Severity	**Informational**
Description	The ManagedNFTManagerUpgradeable contract defines an AlreadyAttached custom error, but this error is never used anywhere in the contract.
Recommendations	Remove the unused error.
Comments / Resolution	Acknowledged.

RouterV2PathProviderUpgradeable

The RouterV2PathProviderUpgradeable contract manages token routing paths for DEX swap operations. It maintains a registry of allowed input tokens and stores multi-hop routes for each output token, enabling strategies to find optimal swap paths during buyback operations.

The owner can add or remove routes via addRouteToToken and removeRouteFromToken, where each route specifies a from token, to token, and stable/volatile flag. Routes are stored per output token and must satisfy that the route's to address matches the token key and from differs from to. The contract prevents duplicate routes with the same from address and stability type.

The getOptimalTokenToTokenRoute function evaluates all registered multi-hop routes plus direct single-hop paths (both stable and volatile) to find the route yielding the highest output amount for a given input. The getAmountOutQuote function provides TWAP-based quoting with configurable granularity for more accurate price estimates.

Privileged Functions

- setAllowedTokenInputRouters
- addRouteToToken
- removeRouterFromToken

Core Invariants:

INV 1: Only the owner can modify routes or the allowed input token whitelist.
INV 2: Each route must have its to field equal to the token it is registered under, and from must differ from to.
INV 3: Duplicate routes with the same from address and stable flag for a given token are rejected.
INV 4: Route paths used in getAmountOutQuote must be continuous, where each segment's to equals the next segment's from.

Issue_105	Typos in variable names and comments
Severity	**Informational**
Description	The RouterV2PathProviderUpgradeable contract contains several typos: 1. Variable name: singelRoute should be singleRoute - used throughout getOptimalTokenToTokenRoute function 2. Variable name: amountOutStabel should be amountOutStable - used throughout getOptimalTokenToTokenRoute function 3. Comment: "creationss" should be "creations" in factory address documentation 4. Comment: "beets" should be "beats" in route selection logic comment
Recommendations	Fix all typos: rename the variable names throughout the function and correct the comment typos.
Comments / Resolution	Acknowledged.

SingelTokenBuybackUpgradeable

The SingelTokenBuybackUpgradeable contract is an abstract base that provides token buyback functionality via DEX swaps. It allows authorized callers to swap any held ERC20 token into a target token (typically NEST) using the RouterV2 infrastructure.

When buybackTokenByV2 is called, the contract queries the RouterV2PathProvider for the optimal route and compares it against any caller-provided custom routes. The route yielding the higher quote is selected. The swap is then executed through the DEX router with slippage protection capped at 4% (400 basis points). The contract validates that custom input routes start with the input token, end with the target token, and only use whitelisted intermediate tokens.

Concrete implementations must override _checkBuybackSwapPermissions to enforce access control and _getBuybackTargetToken to specify which token buybacks should accumulate.

Privileged Functions

- buybackTokenV2

Core Invariants:

INV 1: The input token for a buyback cannot be the same as the target buyback token.
INV 2: Slippage cannot exceed MAX_SLIPPAGE of 400 basis points (4%).
INV 3: Custom input routes must begin with the input token and end with the target token, and all intermediate tokens must be whitelisted.
INV 4: A buyback must result in a non-zero output amount; reverts with RouteNotFound if the quoted output after slippage is zero.

Issue_106	Outdated TWAP price quote in buyback operations enables sandwich attacks beyond slippage protection
Severity	**Medium**
Description	The buybackTokenByV2 function in SingelTokenBuybackUpgradeable calculates the minimum output amount using getAmountOutQuote, which relies on the Pair.quote function. This quote function uses the sample method to retrieve Time-Weighted Average Prices (TWAP) from historical observations stored in the pair contract. The issue is that the TWAP price is a lagging indicator of price. Since observations are recorded every 30 minutes, the TWAP returned can be up to 30+ minutes outdated. The max slippage protection od 4% is applied on this TWAP price, not the current price. Thus the actual slippage can be far higher than 4%, since this is applied on an already stale amount. Due to this, the buyback transactions are highly susceptible to sandwich attacks. Stale prices in volatile market conditions can allow higher than 10% effective slippage, which can be very profitable for MEV bots and gradually eat away at the buyback capital.
Recommendations	Consider setting up an oracle to use the current actual price. Another option would be to at least also consider the current spot price along with the TWAP price when calculating the buyback price, however this option is also subject to MEV attacks since the spot price is highly manipulatable.
Comments / Resolution	Acknowledged.

Issue_107	MAX_SLIPPAGE not configurable
Severity	Low
Description	The contract hard-codes MAX_SLIPPAGE to 400 (4%) and there is no way to change it. 4% slippage is already quite large for normal transactions and allows MEV bots to sandwich the transaction for profit. Having this value hard-coded means that this will always be the case and cannot be further tuned according to market liquidity conditions.
Recommendations	Consider making the MAX_SLIPPAGE value configurable.
Comments / Resolution	Acknowledged.

Issue_108	Multiple typos in SingelTokenBuybackUpgradeable contract, interface, and file name
Severity	Informational
Description	The SingelTokenBuybackUpgradeable contract contains a consistent typo throughout - "Singel" should be "Single":
Recommendations	Fix all the typos in the contract.
Comments / Resolution	Acknowledged.

SingelTokenVirtualRewarderUpgradeable

The SingelTokenVirtualRewarderUpgradeable contract manages virtual reward distribution for users attached to a managed NFT strategy. It tracks virtual balances per tokenId using checkpoints and distributes rewards proportionally based on each token's share of the total supply at each epoch.

When users attach to the strategy, the strategy calls the deposit to record their balance. On detachment, withdrawal reduces the balance, and harvest calculates and returns accumulated rewards. The strategy calls notifyRewardAmount when compounding to add rewards to the current epoch's pool. Reward calculation iterates through all epochs since the user's last harvest, computing their proportional share of each epoch's rewards based on their checkpointed balance versus total supply at that epoch.

The contract does not hold actual reward tokens. Instead, it tracks amounts virtually and returns the calculated reward value to the strategy, which handles the actual token accounting.

Privileged Functions

- deposit
- withdraw
- harvest
- notifyRewardAmount

Core Invariants:

INV 1: Only the linked strategy contract can call deposit, withdraw, harvest, or notifyRewardAmount.
INV 2: The sum of all individual token balances must equal totalSupply at any given time.
INV 3: Rewards for each epoch are distributed proportionally based on the ratio of a token's balance to totalSupply at that epoch.
INV 4: Harvesting sets lastEarnEpoch to the current epoch, preventing double-claiming of rewards for past epochs.

Issue_109	Epoch misalignment leads to lost/incorrect rewards
Severity	**Low**
Description	In the notifyRewardAmount function, the compounded amounts are notified as rewards for the _currentEpoch. Thus if compound is called in epoch 1, rewardsPerEpoch[1] is incremented. *uint256 currentEpoch = _currentEpoch();* *rewardsPerEpoch[currentEpoch] += amount_;* However, when calculating the harvest, a week's offset is used. *return (balance * rewardsPerEpoch[epoch_ + _WEEK]) / supply;* So the rewards for epoch 1 uses the value of rewardsPerEpoch[2]. There are multiple issues with this design. Firstly, the rewardsPerEpoch[2] is not fixed. During epoch 2, any user can donate some NEST tokens to the strategy contract and call compound, which will call notifyRewardAmount and increment rewardsPerEpoch[2]. So later harvesters will get a different amount of rewards than earlier harvesters, since rewardsPerEpoch[2] is changing throughout the epoch. Secondly, once a new epoch starts, if harvest is called before any compound calls, the user loses out on the rewards for one whole week. Say epoch 2 just starts. During the last week, any compound calls only increment rewardsPerEpoch[1], so rewardsPerEpoch[2] is still 0. Now the epoch rolls over, and a user calls harvest. Now, since rewardsPerEpoch[2] = 0, they get 0 rewards for their last epoch. In fact their lastEarnEpoch is now set to 2, so they cannot claim again. Thus they can actually lose rewards. The design is incorrect. notifyRewardAmount should instead allocate rewards to the next epoch, not _currentEpoch. This way users can only claim rewards accrued in the current epoch once it ends.

Recommendations	Change notifyRewardAmount so that it accrues rewards to the next epoch instead of the current one.
Comments / Resolution	Acknowledged.

Issue_110	Notified reward amount in the SingelTokenVirtualRewarder can be lost
Severity	**Medium**
Description	In the SingelTokenVirtualRewarder the notifyRewardAmount updates reward amount for the current epoch where the notified amount resides in the VotingEscrow but if there are no active deposits in that epoch then those notified reward amount is lost forever and stuck in the VotingEscrow since they can't be claimed by anyone.
Recommendations	If the previous epoch had no active deposits then notifyRewardAmount should not notify the rewards.
Comments / Resolution	Acknowledged.

Issue_111	Harvest call would revert if the user has detached many weeks ago
Severity	**Medium**
Description	When a user detaches his tokenId from a managedTokenId then in the SingelTokenVirtualRewarder contract the tokensInfo[tokenId_].lastEarnEpoch his assigned to the current epoch and when in future(which can be many weeks in future) the tokenId is attached again and the user again detaches his tokenId and harvest() is invoked the reward amount is calculated as follows in the _calculateAvailableRewardsAmount: *for (uint256 i; i < notHarvestedEpochCount;) {* *reward += _calculateRewardPerEpoch(tokenId_, startEpoch);* *startEpoch += _WEEK;* *unchecked {* *i++;* *}* *}* We loop through all the epochs within lastEarnEpoch and the currentEpoch and since the amount of weeks can be large the loop might go OOG essentially DoSing the detach mechanism.
Recommendations	Consider adding the ability to harvest a specified number of epochs, instead of harvesting all unharvested epochs. The contract should also correctly update the lastEarnEpoch to the latest harvested epoch.
Comments / Resolution	Acknowledged.

Issue_112	Reward truncation leads to stuck funds
Severity	**Low**
Description	The _calculateRewardPerEpoch function calculates a token's share of rewards for a specific epoch using the following formula: *return (balance * rewardsPerEpoch[epoch_ + _WEEK]) / supply;* Because Solidity truncates the result of division, the sum of rewards distributed to all users for a given epoch will almost always be less than the total rewards notified for that epoch (rewardsPerEpoch). While the error for a single user is small, the cumulative effect across hundreds of users and many epochs leads to a significant accumulation of reward tokens that are no longer claimable by any user through the standard harvest mechanism.
Recommendations	Implement a restricted recovery function that allows an authorized address to withdraw tokens that exceed the sum of all currently claimable rewards.
Comments / Resolution	Acknowledged.

StrategyProxy

The StrategyProxy contract is a minimal proxy that delegates all calls to a strategy implementation address fetched from its factory. The factory address is stored as an immutable set at deployment time.

No issues were identified in this contract.

VirtualRewarderProxy

The VirtualRewarderProxy contract is a minimal proxy that delegates all calls to a virtual rewarder implementation fetched from its factory. The factory address is stored as an immutable set at deployment time.

No issues were identified in this contract.

VirtualRewarderCheckpoints

The VirtualRewarderCheckpoints library is used to manage checkpointed values over time to track the virtual rewards.

Issue_113	Incorrect check in getCheckpointIndex
Severity	**Informational**
Description	In the getCheckpointIndex function, an escape condition is defined in case the checkpointing started after the specified timestamp. This returns 0 if the very first checkpoint's timestamp is already too old. *if (self_[0].timestamp > timestamp_) {* *return 0;* *}* The issue is the the way the library is set up, checkpoint effectively starts from the 1 index. The 0 index is actually just empty uninitialized space, making this check redundant.
Recommendations	Change the check to query index=1 instead.
Comments / Resolution	Acknowledged.

Utils

GetInformationAggregatorUpgradeable

The GetInformationAggregatorUpgradeable contract is a read-only aggregator that batches queries across multiple protocol contracts to provide consolidated vote and gauge information for frontends. It maintains a registry of core protocol addresses (Voter, VotingEscrow, PairFactory, ManagedNFTManager) that the owner can update.

The getTokenIdsVotesPerEpoch function retrieves detailed vote information for a list of token IDs at a specific epoch, including each token's vote power, lock status, attachment status, and per-pool vote weights fetched from internal bribes. The getGeneralVotesPerEpoch function returns aggregate pool-level data for an epoch, including weights, emission allocations, gauge reward rates, and bribe contract totals.

Both functions support pagination via limit and offset parameters and require the epoch parameter to be aligned to weekly boundaries.

Privileged Functions

- updateAddress

Core Invariants:

INV 1: Only the owner can update the registry addresses.
INV 2: Epoch parameters must be aligned to WEEK boundaries (epoch % WEEK == 0).

Issue_114	Potential underflow in getTokenIdsVotesPerEpoch when offset exceeds size
Severity	**Low**
Description	In getTokenIdsVotesPerEpoch, when offset_ >= size, the code sets size = 0 but then unconditionally executes size -= offset_. This will cause an underflow revert when offset_ > 0 and offset_ >= totalCount.
Recommendations	Move size -= offset_ into an else block after the if (offset_ >= size) check, or return early when offset_ >= size.
Comments / Resolution	Acknowledged.

Issue_115	getTokenIdsVotesPerEpoch can return unexpected vote weights
Severity	**Low**
Description	The getTokenIdsVotesPerEpoch function takes in a list of tokenIds and gives a list of their status, including their voting power. The issue is that the voting power of the tokenId is fetched from the bribes system. *uint256 votePower = internalBribe.balanceOfAt(info.tokenId, epoch_);* *if (votePower > 0) {* *tempArray[countVotes].pool = pools[j].pool;* *tempArray[countVotes].weight = votePower;* The bribes system does not store voting power in a per-tokenId basis, instead it stores them on a per-address basis. So if the same owner owns tokenId 1 and 2, getTokenIdsVotesPerEpoch will return the cumulative voting power of both tokens when queried for either of the 2 tokens. This can be unexpected, since the function is set up in a manner where only the info of a single tokenId is returned for other values.
Recommendations	Consider documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_116	Multiple typos in GetInformationAggregator contract
Severity	**Informational**
Description	The GetInformationAggregatorUpgradeable contract contains several typos: 1. "votingEscow" instead of "votingEscrow" 2. "emisisonPerEpoch" instead of "emissionPerEpoch"
Recommendations	Correct all typos to use proper spelling for consistency and clarity.
Comments / Resolution	Acknowledged.

Issue_117	Unused import ISingelTokenVirtualRewarder
Severity	**Informational**
Description	The GetInformationAggregatorUpgradeable contract imports ISingelTokenVirtualRewarder from the nest interfaces but does not use it anywhere in the contract. The interface is not referenced in any function signatures, type declarations, or function calls.
Recommendations	Remove the unused import to reduce contract size and improve code clarity.
Comments / Resolution	Acknowledged.

PairAPIUpgradeable

The PairAPIUpgradeable contract is a read-only API that aggregates pair, gauge, and bribe information for frontend consumption. It queries the Voter, PairFactory, and related contracts to return consolidated data about liquidity pools and user positions.

The getAllPair and getAllCLPair functions return paginated lists of V2 pairs, and V3 concentrated liquidity pools, respectively, including token metadata, reserves, gauge state, emissions, and user-specific balances. The getPair and getCLPair functions return detailed information for individual pools. The getPairBribe function retrieves historical bribe data across epochs for a given pair's external bribe contract.

The owner can update the voter address via setVoter, which also refreshes the pairFactory and underlyingToken references derived from the voter.

Privileged Functions

- setVoter

Core Invariants:

INV 1: Only the owner can update the voter address.
INV 2: Query pagination is capped at MAX_PAIRS (1000) for pair queries and MAX_EPOCHS (200) for bribe epoch queries.

Issue_118	Offset is unused
Severity	**Informational**
Description	The getPairBribe function takes in a value offset. This is used when filling in the _pairEpoch array. *uint i = _offset;* *for (i; i < _offset + _amounts; i++) {* *_supply = bribe.totalSupplyAt(_epochStartTimestamp);* *_pairEpoch[i - _offset].epochTimestamp = _epochStartTimestamp;* The issue is that the offset here is useless. The index i starts from offset, but the actual index used is i-offset, resulting in a normal index starting from 0. Also, the offset is not used for the totalSupplyAt function, since _epochStartTimestamp always starts from bribe.firstBribeTimestamp and not offset from it. Thus the function will always return the same values, no matter the offset.
Recommendations	Consider pushing _epochStartTimestamp by offset number of weeks.
Comments / Resolution	Acknowledged.

Issue_119	Unused events in PairAPIUpgradeable
Severity	**Informational**
Description	The contract declares two events that are never emitted anywhere in the code: - event Owner(address oldOwner, address newOwner); - event WBF(address oldWBF, address newWBF); These appear to be legacy code that was never cleaned up.
Recommendations	Remove the unused events to reduce contract size and improve code clarity.
Comments / Resolution	Acknowledged.

Issue_120	Unused variables in PairAPIUpgradeable
Severity	**Informational**
Description	The _a0Expect and _a1Expect variables in the pairInfo struct are never used in the current logic.
Recommendations	Consider removing these variables or acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_121	Account_gauge_total_weight is never updated
Severity	**Informational**
Description	The _pairInfo.account_gauge_total_weight would always be 0 since the accountGaugeLPTotalWeight is not updated anywhere in the current logic.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

RewardAPIUpgradeable

The RewardAPIUpgradeable contract is a read-only API that aggregates reward and bribe information for frontend consumption. It queries the Voter, gauges, and bribe contracts to return consolidated data about claimable rewards, gauge emissions, and upcoming bribe distributions.

The getAllPairRewards and getAllCLPairRewards functions return paginated reward data for V2 pairs and V3 pools, respectively, including gauge emissions and vote weights for both the total and per-user. The getExpectedClaimForNextEpoch function estimates a token's share of bribe rewards for the upcoming epoch based on current vote balances. The getAvailableRewards and getAvailableBribesRewards functions return currently claimable bribe rewards across specified or all bribe contracts.

The contract maintains a notReward mapping that allows the owner to exclude specific tokens from reward calculations, useful for filtering out invalid or problematic reward tokens.

Privileged Functions

- setVoter
- addNotReward
- removeNotReward

Core Invariants:

INV 1: Only the owner can update the voter address or modify the notReward token exclusion list.
INV 2: Pagination for pair queries is capped at MAX_PAIRS (1000).

Issue_122	Wrong reward tokens used for external bribe
Severity	**Low**
Description	In getAvailableBribesRewards, the external bribe rewards are fetched using rewardTokens from the internal bribe: *address[] memory rewardTokens = IBribe(state.internalBribe).getRewardTokens();* *BribeAvailableRewards memory internalBribeResult = _getBribeRewards(user_, IBribe(state.internalBribe), rewardTokens);* *// ...* *BribeAvailableRewards memory externalBribeResult = _getBribeRewards(user_, IBribe(state.externalBribe), rewardTokens);* *// uses wrong rewardTokens* The external bribe may have different reward tokens than the internal bribe, leading to incorrect or missing reward data being returned.
Recommendations	Fetch reward tokens separately for each bribe. Otherwise always ensure the same list of tokens are used for both bribes and acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_123	Incorrect epoch data fetched in _getNextEpochRewards
Severity	**Low**
Description	In _getNextEpochRewards we fetch the next epoch's starting timestamp which is incorrect since reward data i.e. rewards stored for that epoch would be zero since we only store rewards of the current ongoing epoch.
Recommendations	Consider fetching the correct epoch data.
Comments / Resolution	Acknowledged.

Issue_124	Incorrect boundary check in getAllPairRewards and getAllCLPairRewards
Severity	**Informational**
Description	Lines 169 and 222 use `==` instead of `>=` for boundary checking: *if (i == totPairs) {* *break;* *}* If _offset is provided as a value greater than totPairs, the condition i == totPairs will never be true, and the loop will attempt to access indices out of bounds, causing the call to revert.
Recommendations	Change the condition to `>=`.
Comments / Resolution	Acknowledged.

VeNFTAPIUpgradeable

The VeNFTAPIUpgradeable contract is a read-only API that aggregates veNFT information for frontend consumption. It queries the VotingEscrow, Voter, and ManagedNFTManager contracts to return consolidated data about veNFT positions, their votes, and attachment status.

The getAllNFT function returns paginated veNFT data across all token IDs, while getNFTFromAddress returns all veNFTs owned by a specific user. Each veNFT query includes lock amount, lock end time, voting power, current votes, permanent lock status, and whether the NFT is attached to a managed strategy.

The getAttachedVeNftsRewardInfo function provides detailed reward information for veNFTs attached to managed strategies, including current balances, locked rewards, and historical epoch reward calculations from the virtual rewarder. The getNestApr function calculates the annualized APR for a given strategy rewarder.

Privileged Functions

- setVoter
- setPairAPI
- setManagedNFTManager

Core Invariants:

INV 1: Only the owner can update the voter, pairAPI, or managedNFTManager addresses.
INV 2: Pagination for getAllNFT is capped at MAX_RESULTS (1000).

Issue_125	Incorrect assumption that ownerOf returns address(0) for non-existent tokens
Severity	**Low**
Description	In getAllNFT and getNFTFromIds, the code incorrectly assumes that ve.ownerOf(tokenId) will return address(0) when a token doesn't exist or has been burned. However, according to the ERC721 standard, ownerOf reverts when querying a non-existent token ID. This means these functions will fail completely if they encounter any non-existent or burned token IDs, rather than gracefully skipping them.
Recommendations	Wrap ve.ownerOf calls in try/catch blocks to handle non-existent tokens gracefully.
Comments / Resolution	Acknowledged.

Issue_126	Unused imports in VeNFTAPIUpgradeable contract
Severity	**Informational**
Description	The `VeNFTAPIUpgradeable` contract imports several interfaces that are not used anywhere in the contract: 1. IPairFactory 2. IPair 3. IGauge 4. IBribe These interfaces are not referenced in any function signatures, type declarations, or function calls within the contract.
Recommendations	Remove the unused imports to reduce contract size and improve code clarity.
Comments / Resolution	Acknowledged.

Issue_127	Redundant declarations not used in VeNFTAPIUpgradeable contract
Severity	**Informational**
Description	The VeNFTAPIUpgradeable contract contains several redundant declarations that are not used anywhere in the contract: 1. Struct Reward: Defined but never used. It's only referenced by AllPairRewards struct, which is also unused. 2. Constant MAX_PAIRS: Defined as 30 but never referenced in any function or validation. 3. State variable underlyingToken: Set in initialize but never read or used in any function. 4. Mapping notReward: Set to true for address(0) in initialize but never read or used. This appears to be leftover code from RewardAPI contract. 5. State variable pairAPI: Has a setter function but is never read or used in any function. 6. Struct AllPairRewards: Defined but never used anywhere in the contract. 7. Function setPairAPI: Sets pairAPI which is unused, making this function redundant.
Recommendations	Remove all unused structs, constants, state variables, mappings, and functions to reduce contract size, improve code clarity, and save gas.
Comments / Resolution	Acknowledged.

Vesting

MinimalLinearVestingUpgradeable

The MinimalLinearVestingUpgradeable contract manages linear token vesting with a configurable start time and duration. The owner sets wallet allocations before vesting begins, and users can claim their tokens proportionally as they vest over time.

When setting allocations via setWalletsAllocation, the contract automatically adjusts its token balance by transferring excess tokens back to the owner or pulling additional tokens from the owner to match the new totalAllocated amount. Once the claim phase starts (block.timestamp >= startTimestamp), allocations and vesting parameters become immutable. Users call claim to receive their vested portion, calculated as a linear proportion of elapsed time over the total duration minus any previously claimed amount.

Privileged Functions

- setWalletsAllocation
- setVestingParams

Core Invariants:

INV 1: Allocations and vesting parameters cannot be modified once the claim phase has started.
INV 2: The contract's token balance always equals totalAllocated; adjustments are made automatically when allocations change.
INV 3: Users can only claim tokens that have vested (proportional to elapsed time) minus what they have already claimed.
INV 4: After the full duration has elapsed, 100% of a wallet's allocation becomes claimable.

Issue_128	Inconsistent timestamp check
Severity	**Low**
Description	Function isClaimPhase returns true when start<=block.timestamp, indicating that the claiming phase is on. The modifier onlyNotDuringClaimPhase reverts when startTimestamp < block.timestamp, allowing execution when the claim phase is not on. The issue is that at start==block.timestamp, isClaimPhase returns true indicating that claim phase is on but onlyNotDuringClaimPhase also allows the function to execute, indicating that the claim phase is not on. This is inconsistent since the two functions behave differently at the same timestamp.
Recommendations	Consider changing the check to startTimestamp <= block.timestamp in onlyNotDuringClaimPhase to maintain consistency.
Comments / Resolution	Acknowledged.